

MCQ COMPLETE MASTER GUIDE

Learn the concepts first. Then train on the traps.

EXAM-ALIGNED	MCQ ONLY	DEEP CONCEPTS	MULTIPLE QUESTION STYLES
34 MCQs / 40 marks	384 practice MCQs	Topic notes + triggered traps	Direct • Scenario • Trap • Comparison • Reasoning

Important: This edition is intentionally MCQ-oriented. It removes the separate PySpark coding-question section. The supplied exam bifurcation still guides topic priority, but this book is designed to build the conceptual depth needed to answer differently-worded MCQs.

Source basis: your Milestone 2 preparation handbook, your earlier 100-MCQ set, the expanded MCQ bank, and the exact bifurcation shown in your screenshot.

1. Exact Exam Blueprint You Gave

The table below preserves the MCQ portion of the supplied bifurcation. The separate Easy/Medium PySpark coding marks are deliberately excluded from this book, as requested.

Section	Difficulty	MCQs	Marks	Priority
Java Fundamentals, Core Java & Collections	Easy	4	4	HIGH
Java OOP	Easy	4	4	HIGH
Java File I/O & APIs	Easy	3	3	CORE
Scala Basics & Data Structures	Easy	3	3	CORE
Scala RDDs	Easy	4	4	HIGH
Scala File I/O	Easy	2	2	CORE
Advanced Spark	Medium	2	4	HIGH
Scala Transformations	Medium	2	4	HIGH
Scala DataFrames	Medium	2	4	HIGH
Spark Streaming & Fault Tolerance	Difficult	2	4	HIGH
Java OOPs / APIs / Collections	Difficult	2	4	HIGH

Scoring insight: the two-mark Medium/Difficult areas deserve disproportionate preparation because a single question there is worth more than an Easy one-mark MCQ. The guide therefore gives Advanced Spark, Transformations, DataFrames, Streaming and the difficult Java integration section deeper treatment.

2. How to Study This Book

Pass 1 — Learn

Read the topic notes and the “Exam Triggers” before touching the MCQs. The aim is to understand what each mechanism solves, not to memorize answer letters.

Pass 2 — Practice

Attempt the MCQs without looking at the answer. Mark every question where you guessed, even if you happened to be correct.

Pass 3 — Diagnose

Read the explanation and the trap. Your goal is to know why the correct option is correct and why the tempting alternatives are wrong.

Pass 4 — Mix

Use the integrated sections at the end. These deliberately combine Java, Scala and Spark concepts so that the wording—not the section title—has to guide you.

Pass 5 — Final revision

Use the comparison sheets and trigger maps. If you can explain the difference between every paired concept without looking, you are in strong shape for conceptual MCQs.

3. Java Fundamentals, Core Java & Collections

Easy difficulty • 4 official MCQs • 4 official marks • Expanded practice below

What this section is really testing

This section is the foundation for Java MCQs. The handbook emphasizes JVM execution, stack/heap reasoning, collections, generics, equality/hashing, iterators and short-circuit operators. The exam depth is conceptual: you should be able to read a short scenario and identify the correct Java mechanism without relying on memorized answer letters.

Deep Concept Notes

JDK, JRE and JVM

The JDK is the development kit: it includes tools needed to compile and build Java programs, including `javac`. The JVM is the runtime engine that loads and executes Java bytecode. The JRE is the runtime environment conceptually surrounding the JVM and runtime libraries. The important exam chain is: source `.java` → compiler/`javac` → bytecode `.class` → JVM executes bytecode. Exam trigger: “compile/develop” → JDK; “execute bytecode” → JVM; “.class” → bytecode.

Stack vs Heap

A method call creates a stack frame containing local variables and references associated with that call. Objects created with `new` are generally allocated on the heap. A reference may be stored in a stack frame while the object it refers to lives on the heap. MCQs often deliberately mix up the location of the reference and the object. Exam trigger: “`new Customer()` / object allocation” → heap; “local variable / method frame” → stack.

Collections: List, Set, Map

List models an ordered sequence and permits duplicates. Set models uniqueness. Map stores key/value associations rather than being a simple collection of values. Choosing the structure is usually driven by the requirement: preserve duplicates/order → List; prevent duplicate elements → Set; lookup a value by a key → Map.

HashMap and duplicate keys

A Map has one mapped value for a given key. Calling `put` with an existing key replaces the value associated with that key. The key itself is not duplicated. Hash-based collections depend on consistent `equals()` and `hashCode()` behavior: equal objects should produce the same hash code.

Generics

Generics provide compile-time type safety. `List<String>` communicates that the list is intended to contain strings. Wildcards describe variance-style constraints: `List<? extends Number>` means the list's element type is some unknown subtype of `Number` and is primarily useful when reading values as `Number`. Avoid thinking of generics as merely documentation; they change what the compiler permits.

Iterator and safe removal

Removing elements directly from a collection while traversing it can cause structural-modification problems. `Iterator.remove()` is designed to remove the element most recently returned by `next()` while keeping the iterator's state consistent. In scenario questions, “remove while iterating” is a strong signal for `Iterator.remove()`.

Autoboxing

Java collections store reference types, so primitive values such as `int` are automatically boxed into wrapper objects such as `Integer` when needed. This automatic conversion is called autoboxing. The reverse conversion is unboxing.

Short-circuit boolean operators

With A && B, if A is false, B is not evaluated. With A || B, if A is true, the right side is not evaluated. This is why a null check can safely appear before a method call: `user != null && user.isActive()`. Exam questions often test the evaluation order rather than the final boolean value.

Exam Trigger Map

- JDK is for development; JVM is what executes bytecode.
- An object and the reference to it are not the same thing.
- Set uniqueness is different from Map key/value association.
- Same HashMap key does not create a second mapping; the value is replaced.
- Generics give compile-time type checking; they are not runtime storage containers.
- Do not use direct collection removal casually during iterator traversal.
- && and || can skip the right-hand side; this matters for null checks.

FAST SIGNAL	THINK OF
Trap	JDK is for development; JVM is what executes bytecode.
Trap	An object and the reference to it are not the same thing.
Trap	Set uniqueness is different from Map key/value association.
Trap	Same HashMap key does not create a second mapping; the value is replaced.
Trap	Generics give compile-time type checking; they are not runtime storage containers.
Trap	Do not use direct collection removal casually during iterator traversal.
Trap	&& and can skip the right-hand side; this matters for null checks.

MCQ Training Bank — 32 Questions

Question styles are intentionally mixed: direct recall, scenario-based, conceptual reasoning, comparison, trap questions, classification, cardinality/output reasoning and performance/architecture reasoning where appropriate.

Q1 [Original Practice]

A trainee installs Java to compile and run a program. Which component contains the Java compiler (javac)?

- A) JDK
- B) JRE
- C) JVM only
- D) Garbage Collector

✓ **Answer: A) JDK**

Why: The JDK includes development tools such as javac, while the JRE provides the runtime environment. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q2 [Original Practice]

After a Java source file is compiled successfully, what does the compiler normally produce for the JVM to execute?

- A) A native .exe file
- B) Java bytecode in a .class file
- C) A SQL script
- D) A compressed JAR only

✓ **Answer: B) Java bytecode in a .class file**

Why: Java compilation normally produces bytecode in .class files, which the JVM executes. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q3 [Original Practice]

A training app must store unique course codes and should automatically prevent duplicates. Which collection type best matches this requirement?

- A) List
- B) Set
- C) Queue
- D) Array

✓ **Answer: B) Set**

Why: A Set stores unique elements and is the natural choice when duplicates must be prevented. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q4 [Original Practice]

A Java method creates a local int variable and also creates a new Customer object. In the usual memory model, where is the object primarily allocated?

- A) Stack only
- B) Heap
- C) CPU register only
- D) Source-code area

✓ **Answer: B) Heap**

Why: Objects are generally allocated on the heap; local variables and references are associated with method stack frames. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q5 [Original Practice]

A method should accept a List of String values and prevent callers from accidentally adding Integer values at compile time. Which declaration is most suitable?

- A) List values
- B) List<Object> values
- C) List<String> values
- D) Object values

✓ **Answer: C) List<String> values**

Why: Generics provide compile-time type safety. List<String> restricts elements to strings. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q6 [Original Practice]

A List<Integer> receives the primitive value 25 using list.add(25). Why is this allowed?

- A) Java converts the List into an int array
- B) Autoboxing converts int to Integer
- C) The compiler ignores the type
- D) Integer is a primitive type

✓ **Answer: B) Autoboxing converts int to Integer**

Why: Autoboxing automatically converts a primitive int to its corresponding Integer wrapper when required. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q7 [Original Practice]

A HashMap stores employee details using employeeId as the key. What happens when put() is called again with the same key and a new value?

- A) A duplicate key is stored
- B) The existing value for that key is replaced
- C) The map becomes a Set
- D) The key is automatically deleted

✓ **Answer: B) The existing value for that key is replaced**

Why: A Map keeps one value per key. Putting the same key again replaces the mapped value. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q8 [Original Practice]

A trainee must remove selected elements while iterating through an ArrayList and wants to avoid unsafe structural modification. Which approach is appropriate?

- A) Call list.remove() from every foreach iteration
- B) Use Iterator and its remove() method after next()
- C) Convert the list to a String first
- D) Change the list reference to null

✓ **Answer: B) Use Iterator and its remove() method after next()**

Why: Iterator.remove() is designed for safe removal of the current element during iterator-based traversal. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q9 [Original Practice]

A condition is written as user != null && user.isActive(). If user is null, why does the expression avoid a NullPointerException from isActive()?

- A) && evaluates the right side first
- B) && skips the right side when the left side is false
- C) isActive() is converted to static
- D) null is treated as an active user

✓ **Answer: B) && skips the right side when the left side is false**

Why: The && operator short-circuits. If the left operand is false, Java does not evaluate the right operand. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q10 [Direct]

Which component is primarily responsible for executing Java bytecode at runtime?

- A) JDK
- B) JVM
- C) javac source compiler
- D) JAR

✓ **Answer: B) JVM**

Why: The JVM executes bytecode; the JDK supplies development tools. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q11 [Direct]

Which file normally contains Java bytecode after compilation?

- A) .java
- B) .class
- C) .sql
- D) .exe

✓ **Answer: C) .sql**

Why: The Java compiler normally emits .class files containing bytecode. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q12 [Direct]

Which collection is the natural choice when uniqueness is the primary requirement?

- A) List
- B) Set
- C) Map
- D) Iterator

✓ **Answer: C) Map**

Why: Set models a collection of unique elements. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q13 [Direct]

What does autoboxing do?

- A) Converts a wrapper to a primitive
- B) Converts a primitive to its wrapper automatically
- C) Converts Java to bytecode
- D) Converts a Set to a List

✓ **Answer: C) Converts Java to bytecode**

Why: Autoboxing converts values such as int to Integer when an object is required. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q14 [Direct]

Which statement about a HashMap is correct?

- A) Keys may have unlimited duplicate mappings
- B) A later put for an equal key replaces its mapping
- C) Values must be unique
- D) HashMap is a subtype of Set

✓ **Answer: C) Values must be unique**

Why: A map has one current mapping per key; putting the same key updates its value. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q15 [Conceptual]

A developer needs an ordered sequence in which duplicate customer names are allowed. Which abstraction fits best?

- A) Set
- B) List
- C) Map
- D) HashMap

✓ **Answer: C) Map**

Why: List preserves sequence semantics and permits duplicates. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q16 [Conceptual]

A custom object is placed in a HashSet, but equals() is overridden without a compatible hashCode(). What risk exists?

- A) The set becomes ordered
- B) Logical duplicate handling can become inconsistent
- C) The object becomes immutable
- D) The JVM refuses to start

✓ **Answer: C) The object becomes immutable**

Why: Hash-based collections depend on a consistent equals/hashCode contract for logical equality. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q17 [Conceptual]

Why do generics reduce the need for explicit casts in collections?

- A) They move all objects to the heap
- B) They provide compile-time type information
- C) They disable inheritance
- D) They make every collection immutable

✓ **Answer: C) They disable inheritance**

Why: A declaration such as `List<String>` lets the compiler enforce and know the element type. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q18 [Conceptual]

Which statement best describes `List<? extends Number>`?

- A) It can only contain `String`
- B) It represents a list of `Number` or an unknown subtype of `Number`
- C) It can only contain `Object`
- D) It is a primitive array

✓ **Answer: C) It can only contain Object**

Why: `extends` gives an upper bound: the actual element type is `Number` or some subtype. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q19 [Conceptual]

Why is `Iterator.remove()` preferred over calling `collection.remove()` directly during a for-each traversal?

- A) It coordinates removal with the iterator traversal
- B) It sorts the collection first
- C) It copies the collection to disk
- D) It converts values to primitives

✓ **Answer: B) It sorts the collection first**

Why: `Iterator.remove()` is designed to remove the last returned element consistently with iterator state. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q20 [Scenario]

A service keeps one shared request counter for every instance of a class. Which member kind represents class-level shared state?

- A) Instance field
- B) static field
- C) local variable
- D) constructor parameter

✓ **Answer: C) local variable**

Why: A static field belongs to the class rather than each individual object. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q21 [Scenario]

A trainee sees byte a=10 and byte b=20, then int c=a+b. Why is int appropriate for c?

- A) byte arithmetic is promoted to int
- B) byte is a reference type
- C) JVM always converts numbers to double
- D) addition returns byte

✓ **Answer: B) byte is a reference type**

Why: Java numeric promotion promotes byte/short operands to int for arithmetic expressions. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q22 [Scenario]

An application evaluates user != null && user.isActive(). What happens when user is null?

- A) isActive() still runs
- B) The right side is skipped
- C) user becomes an empty object
- D) JVM throws an exception before evaluating &&

✓ **Answer: C) user becomes an empty object**

Why: && short-circuits when its left side is false, so isActive() is not evaluated. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q23 [Scenario]

A developer writes $2 + 3 * 4$ and gets 14. Which rule explains the result?

- A) Addition has higher precedence
- B) Multiplication has higher precedence than addition
- C) Java evaluates right to left only
- D) Parentheses are implicit around $2+3$

✓ **Answer: C) Java evaluates right to left only**

Why: Operator precedence groups multiplication before addition. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q24 [Scenario]

A team needs to remove negative integers from an ArrayList while traversing it. Which pattern is safest?

- A) foreach plus list.remove()
- B) Iterator, next(), then iterator.remove()
- C) Set the list to null
- D) Convert every value to String first

✓ **Answer: C) Set the list to null**

Why: Iterator.remove() coordinates structural removal with traversal. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q25 [Scenario]

A class stores a serialized session object but must exclude a temporary token from default serialization. Which keyword is relevant?

- A) transient
- B) static
- C) volatile
- D) synchronized

✓ **Answer: B) static**

Why: transient marks a field to be skipped by default Java serialization. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q26 [Conceptual]

Two references point to the same Customer object created with new. What is the key memory-model idea?

- A) There are necessarily two heap objects
- B) Multiple references can refer to one heap object
- C) The object must live on the stack only
- D) References cannot be copied

✓ **Answer: C) The object must live on the stack only**

Why: A reference variable can point to an object allocated on the heap, and multiple references can refer to that same object. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q27 [Conceptual trap]

A team says “the JVM compiles .java files.” Which correction is most accurate?

- A) Correct; JVM is the compiler
- B) javac in the JDK normally compiles source to bytecode
- C) JRE compiles source and JVM stores it
- D) Garbage Collector compiles source

✓ **Answer: B) javac in the JDK normally compiles source to bytecode**

Why: The JDK supplies development tools such as javac. The JVM's job is to execute bytecode, not normally compile Java source files. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property before selecting the option.

Q28 [Scenario]

A requirement says: “store employee IDs, never allow duplicate IDs, and membership testing matters.” Which structure best matches the requirement?

- A) List
- B) Set
- C) Map with arbitrary values
- D) Array only

✓ **Answer: B) Set**

Why: The primary requirement is uniqueness, which is modeled directly by Set. A List permits duplicates. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q29 [Trap]

Two objects are logically equal but their hashCode() values differ. What is the key concern for a HashSet?

- A) Nothing; hashCode is irrelevant
- B) Hash-based lookup/uniqueness can behave incorrectly
- C) The objects become primitives
- D) The set automatically sorts them

✓ **Answer: B) Hash-based lookup/uniqueness can behave incorrectly**

Why: Hash-based collections rely on consistent equals/hashCode behavior. Equal objects should have the same hash code. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: compare the tempting alternatives against the exact definition.

Q30 [Generics reasoning]

Which statement best describes List<? extends Number>?

- A) It can only contain String
- B) It is a list of some unknown subtype of Number
- C) It is a list that accepts every Object for insertion
- D) It means the list itself is immutable

✓ **Answer: B) It is a list of some unknown subtype of Number**

Why: The wildcard says the element type is some unknown type that extends Number. It does not mean immutability. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property before selecting the option.

Q31 [Scenario]

Why is `x != null && x.isActive()` a common safe pattern?

- A) Java evaluates the right side first
- B) && skips the right side when the left side is false
- C) null is converted to false automatically
- D) isActive becomes static

✓ **Answer: B) && skips the right side when the left side is false**

Why: Short-circuit && evaluates the right operand only if the left operand is true. Therefore isActive is not called when x is null. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q32 [Direct]

Which statement about autoboxing is correct?

- A) Integer is a primitive
- B) int can be automatically converted to Integer when a wrapper is required
- C) Java collections store primitives directly
- D) Autoboxing converts String to int

✓ **Answer: B) int can be automatically converted to Integer when a wrapper is required**

Why: Autoboxing is automatic primitive-to-wrapper conversion, such as int to Integer. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

4. Java OOP

Easy difficulty • 4 official MCQs • 4 official marks • Expanded practice below

What this section is really testing

The handbook treats OOP as relationship and dispatch reasoning rather than vocabulary memorization. The most important distinctions are encapsulation vs abstraction, inheritance vs composition, overloading vs overriding, compile-time selection vs runtime dispatch, interface vs abstract class, and the roles of `this`/`super`/`final`.

Deep Concept Notes

Encapsulation

Encapsulation means keeping internal state controlled and exposing operations that enforce valid changes. A private balance field plus deposit/withdraw methods is a classic example. The deeper exam idea is not “getter/setter = encapsulation”; it is controlled access to state and preservation of object invariants.

Inheritance and “is-a”

Inheritance represents a specialization relationship: Dog is an Animal. A child class inherits accessible behavior/state from its parent and can add or override behavior. Do not use inheritance merely because two classes share some code; the relationship should be semantically valid.

Composition and “has-a”

Composition models ownership/containment: Car has an Engine. It is often preferred when a component should be replaceable or when inheritance would create unnecessary coupling. A scenario mentioning “contains”, “uses”, or “has a” should make you consider composition.

Overloading vs Overriding

Overloading means same method name with different parameter lists; the compiler selects an applicable overload.

Overriding means a subclass supplies a replacement implementation for an inherited instance method; runtime dispatch can select the child implementation based on the actual object. Return type alone does not create a valid overload.

this and super

`this` refers to the current object and is commonly used to distinguish fields from parameters or invoke another constructor in the same class. `super` refers to the immediate parent and can invoke a parent constructor or access parent behavior.

Constructor questions often test whether `this(...)` or `super(...)` must appear first.

Interfaces and abstract classes

An interface defines a contract that implementing classes agree to satisfy. An abstract class can combine shared state/implementation with abstract methods and cannot be instantiated directly. At this assessment depth, choose an interface when the emphasis is a capability/contract across potentially unrelated classes; choose an abstract class when closely related classes should share implementation or state.

Polymorphism and runtime dispatch

A parent reference can refer to a child object. If an instance method is overridden, the implementation belonging to the runtime object is selected. This is the core idea behind runtime polymorphism. The declared reference type controls what members are accessible at compile time, while the runtime object matters for overridden method dispatch.

final and SRP

A final method cannot be overridden. A final variable cannot be reassigned after initialization, and a final class cannot be subclassed. The Single Responsibility Principle (SRP) aims to prevent classes from accumulating unrelated

responsibilities and therefore multiple independent reasons to change.

Exam Trigger Map

- Overloading = different parameters; overriding = child replacement of inherited behavior.
- Return type alone is not enough to overload a method.
- has-a → composition; is-a → inheritance.
- final method cannot be overridden.
- Runtime object determines an overridden instance method's implementation.
- Interface = contract; abstract class can provide shared state/implementation.

FAST SIGNAL	THINK OF
Trap	Overloading = different parameters; overriding = child replacement of inherited behavior.
Trap	Return type alone is not enough to overload a method.
Trap	has-a → composition; is-a → inheritance.
Trap	final method cannot be overridden.
Trap	Runtime object determines an overridden instance method's implementation.
Trap	Interface = contract; abstract class can provide shared state/implementation.

MCQ Training Bank — 32 Questions

Question styles are intentionally mixed: direct recall, scenario-based, conceptual reasoning, comparison, trap questions, classification, cardinality/output reasoning and performance/architecture reasoning where appropriate.

Q1 [Original Practice]

A BankAccount class keeps balance private and exposes deposit() and withdraw() methods. Which OOP principle is being applied?

- A) Encapsulation
- B) Inheritance
- C) Method overloading
- D) Serialization

✓ **Answer: A) Encapsulation**

Why: Encapsulation hides internal state and provides controlled access through methods. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q2 [Original Practice]

Car extends Vehicle. What relationship does this represent?

- A) Car has-a Vehicle
- B) Car is-a Vehicle
- C) Vehicle is a method of Car
- D) Car and Vehicle are unrelated

✓ **Answer: B) Car is-a Vehicle**

Why: Inheritance models an is-a relationship: a Car is a specialized Vehicle. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q3 [Original Practice]

A class contains print(int value) and print(String value). What concept does this demonstrate?

- A) Overriding
- B) Overloading
- C) Abstraction only
- D) Constructor chaining

✓ **Answer: B) Overloading**

Why: Overloading uses the same method name with different parameter lists in the same class hierarchy. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q4 [Original Practice]

A parent class defines calculateFee(), and a child class provides its own implementation with the same signature. Which concept is used?

- A) Method overriding
- B) Method overloading
- C) Autoboxing
- D) Serialization

✓ **Answer: A) Method overriding**

Why: A subclass can override an inherited method to provide specialized runtime behavior. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q5 [Original Practice]

A child constructor must initialize fields defined by its parent class using the parent constructor. Which keyword is normally used?

- A) this
- B) super
- C) final
- D) static

✓ **Answer: B) super**

Why: super(...) invokes a parent-class constructor. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q6 [Original Practice]

Several unrelated classes must guarantee that they provide a process() method, while each class can implement it differently. Which Java construct best expresses this contract?

- A) Interface
- B) Private constructor
- C) Array
- D) Package only

✓ **Answer: A) Interface**

Why: An interface defines a common contract that multiple unrelated classes can implement. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q7 [Original Practice]

An OrderService uses a PaymentGateway object to process payments, but OrderService is not a type of PaymentGateway. Which design relationship is most appropriate?

- A) Inheritance
- B) Composition
- C) Overloading
- D) Casting

✓ **Answer: B) Composition**

Why: Composition models a has-a/uses-a relationship and avoids forcing an incorrect is-a hierarchy. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q8 [Original Practice]

Inside a subclass, which expression refers to the current object, and which refers to the parent-class implementation?

- A) this and super
- B) super and this
- C) static and final
- D) new and class

✓ **Answer: A) this and super**

Why: this refers to the current object; super refers to the parent-class portion or implementation. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q9 [Original Practice]

Vehicle v = new Car(); is followed by v.move(), and Car overrides move(). Which implementation executes?

- A) Vehicle.move() always
- B) Car.move() because overridden instance methods use runtime dispatch
- C) Both methods execute automatically
- D) The code cannot compile

✓ **Answer: B) Car.move() because overridden instance methods use runtime dispatch**

Why: For overridden instance methods, Java chooses the implementation based on the runtime object type, here Car. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q10 [Direct]

What is the defining feature of method overloading?

- A) Same name with different parameter lists
- B) Same signature in parent and child
- C) Different return type only
- D) Same method copied to another object

✓ **Answer: B) Same signature in parent and child**

Why: Overloading provides multiple applicable signatures distinguished by parameters. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q11 [Direct]

What does method overriding support in Java?

- A) Runtime dispatch of specialized child behavior
- B) Compile-time type inference only
- C) Automatic serialization
- D) Primitive boxing

✓ **Answer: B) Compile-time type inference only**

Why: An overridden instance method can be selected according to the runtime object type. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q12 [Direct]

Which keyword invokes a parent constructor?

- A) this
- B) super
- C) final
- D) static

✓ **Answer: C) final**

Why: super(...) invokes a constructor of the immediate parent class. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q13 [Direct]

Which relationship is modeled by composition?

- A) is-a
- B) has-a/uses-a
- C) compiles-to
- D) equals-a

✓ **Answer: C) compiles-to**

Why: Composition models containment or collaboration rather than subtype inheritance. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q14 [Direct]

What does final on an instance method mean?

- A) It cannot be overridden
- B) It cannot be called
- C) It becomes static
- D) It must return final

✓ **Answer: B) It cannot be called**

Why: A final method cannot be overridden by subclasses. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q15 [Conceptual]

A parent reference points to a child object, and the child overrides an instance method. Which implementation is normally selected?

- A) Parent implementation always
- B) Child implementation at runtime
- C) Both automatically
- D) Neither

✓ **Answer: C) Both automatically**

Why: Runtime dispatch uses the actual object for overridden instance methods. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q16 [Conceptual]

Why is changing only the return type insufficient to create a legal overload?

- A) Overload resolution uses parameter lists, not return type alone
- B) Java ignores return types everywhere
- C) Return types are never compiled
- D) Methods cannot return values

✓ **Answer: B) Java ignores return types everywhere**

Why: Two methods with the same name and parameters cannot be overloaded solely by changing return type. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q17 [Conceptual]

Which choice best represents an interface?

- A) A contract that implementing classes agree to provide
- B) A heap allocation instruction
- C) A mandatory database transaction
- D) A concrete object instance

✓ **Answer: B) A heap allocation instruction**

Why: Interfaces express capabilities/contracts and allow different implementations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q18 [Conceptual]

Why might an abstract class be chosen instead of an interface?

- A) It can provide shared state and implemented behavior to related subclasses
- B) It can always be instantiated directly
- C) It cannot have fields
- D) It only supports unrelated classes

✓ **Answer: B) It can always be instantiated directly**

Why: An abstract class can combine abstract operations with shared implementation/state. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q19 [Conceptual]

What does SRP primarily ask a class to have?

- A) One primary reason to change
- B) One field only
- C) Exactly one method
- D) No dependencies

✓ **Answer: B) One field only**

Why: Single Responsibility Principle focuses on cohesive responsibility and one primary reason to change. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q20 [Scenario]

A Car contains an Engine field and delegates engine work to it. Which relationship is being modeled?

- A) Inheritance
- B) Composition
- C) Overloading
- D) Constructor hiding

✓ **Answer: C) Overloading**

Why: A Car has/uses an Engine, so composition is the natural relationship. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q21 [Scenario]

A subclass constructor needs to initialize parent state before its own constructor body. Which call is relevant?

- A) super(...)
- B) this(...) after arbitrary statements
- C) final(...)
- D) static(...)

✓ **Answer: B) this(...) after arbitrary statements**

Why: The parent constructor is invoked with super(...), and an explicit constructor call is first. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q22 [Scenario]

A class exposes private balance but only allows deposits through a method that validates positive amounts. What principle is illustrated?

- A) Encapsulation
- B) Overloading
- C) Inheritance
- D) Autoboxing

✓ **Answer: B) Overloading**

Why: Private state plus controlled operations protects invariants and is encapsulation. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q23 [Scenario]

An interface `Payment` has implementations `CardPayment` and `UpiPayment`. A variable typed `Payment` refers to either implementation. What design benefit is illustrated?

- A) Runtime polymorphism and interchangeable implementations
- B) Numeric promotion
- C) File traversal
- D) Serialization compatibility

✓ **Answer: B) Numeric promotion**

Why: A common abstraction allows callers to use different concrete implementations polymorphically. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q24 [Scenario]

A public method in a parent class is overridden by a child. Can the child make the overriding method private?

- A) Yes
- B) No, visibility cannot be narrowed
- C) Only if final
- D) Only for constructors

✓ **Answer: C) Only if final**

Why: An overriding method cannot reduce inherited visibility; public must remain at least public. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q25 [Scenario]

A subclass wants to call another constructor in the same class before initializing the object body. Which keyword is used?

- A) `this(...)`
- B) `super(...)`
- C) `parent(...)`
- D) `current(...)`

✓ **Answer: B) `super(...)`**

Why: `this(...)` delegates to another constructor in the same class and must be the first statement. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q26 [Conceptual]

Which statement about final is correct?

- A) final class prevents extension
- B) final method prevents object creation
- C) final variable makes every referenced object deeply immutable
- D) final always means static

✓ **Answer: B) final method prevents object creation**

Why: A final class cannot be subclassed. final has different effects on variables, methods, and classes. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q27 [Scenario]

A parent variable refers to a child object and an instance method is overridden. Which behavior is expected?

- A) Parent implementation is always selected
- B) Child override can be selected at runtime
- C) The method must be static
- D) The program must fail

✓ **Answer: B) Child override can be selected at runtime**

Why: Overridden instance methods use runtime dispatch, so the actual child object's implementation can execute. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q28 [Relationship]

Which design is the clearest example of composition?

- A) Dog extends Animal
- B) Car contains an Engine
- C) Child extends Parent
- D) Square overrides Shape

✓ **Answer: B) Car contains an Engine**

Why: Composition models a has-a relationship. A Car has an Engine. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property before selecting the option.

Q29 [Trap]

Which change would NOT create a valid overload by itself?

- A) Change parameter type
- B) Add a parameter
- C) Change only the return type
- D) Change parameter order with distinct types

✓ **Answer: C) Change only the return type**

Why: Overloading is distinguished by the parameter list. Return type alone is insufficient. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: compare the tempting alternatives against the exact definition.

Q30 [Scenario]

A class needs shared state and some implemented methods for closely related subclasses. Which is the stronger conceptual fit?

- A) Abstract class
- B) Primitive
- C) Iterator
- D) Set

✓ **Answer: A) Abstract class**

Why: An abstract class can contain state and implemented behavior while also requiring subclasses to implement abstract methods. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q31 [Direct]

What does `super(...)` in a constructor primarily refer to?

- A) Current object's constructor
- B) Immediate parent's constructor
- C) Any unrelated object's constructor
- D) JVM constructor

✓ **Answer: B) Immediate parent's constructor**

Why: `super(...)` invokes a constructor of the immediate parent class. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q32 [Scenario]

A class exposes only controlled methods for changing a private balance and validates withdrawals. What OOP idea is strongest?

- A) Encapsulation
- B) Overloading
- C) Inheritance
- D) Serialization

✓ **Answer: A) Encapsulation**

Why: Encapsulation protects internal state and exposes controlled behavior that can enforce invariants. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

5. Java File I/O & APIs

Easy difficulty • 3 official MCQs • 3 official marks • Expanded practice below

What this section is really testing

This section combines the handbook's Java I/O, streams, regex, date/time, HTTP client, serialization and directory traversal material. The key is matching the requirement to the correct API family and recognizing resource-management patterns.

Deep Concept Notes

Byte streams vs character streams

Byte streams such as `FileInputStream/FileOutputStream` are appropriate for raw binary data such as image bytes. Character-oriented APIs such as `Reader/Writer` are intended for text. `PrintWriter` is convenient for writing formatted text. The question usually gives the data type as the clue.

try-with-resources

`try-with-resources` automatically closes resources that implement `AutoCloseable/Closeable`. It is preferred for files, streams and JDBC resources because it makes cleanup reliable even when an exception occurs. If the scenario emphasizes avoiding leaks or guaranteed closure, think `try-with-resources`.

Java Streams API

The Java Streams API supports declarative pipelines such as `filter` → `map` → `toList/collect`. Intermediate operations build a pipeline; terminal operations consume it. Do not confuse `java.util.stream.Stream` with `FileInputStream` or Spark Streaming: they are different APIs.

Regex Pattern and Matcher

`Pattern` represents a compiled regular expression and `Matcher` applies it to input. `matches()` attempts to match the entire input, making it a natural choice for validation. `find()` searches for matching subsequences within a larger input. The words “validate the whole ID” strongly point to `matches()`; “find occurrences in text” points to `find()`.

java.time

`LocalDate` represents a date without time or timezone. `LocalDateTime` adds time-of-day but still has no timezone. `ZonedDateTime` carries timezone context. Match the type to the requirement rather than memorizing names in isolation.

transient and serialization

A transient field is skipped by default Java object serialization. It can be useful for sensitive, derived or non-serializable state that should not be part of the default serialized form. `transient` is not an access modifier and does not mean “private”.

Files.walk and directory traversal

`Files.walk` can traverse a directory tree and returns a stream of paths. Because the returned stream represents a resource-backed traversal, it should be handled with appropriate closure, commonly through `try-with-resources`. Typical use cases include finding files, filtering extensions and calculating sizes.

HTTP Client

The modern `java.net.http` API uses `HttpClient`, `HttpRequest` and `HttpResponse`. At this exam depth, recognize the basic request → send → response flow rather than advanced networking configuration.

Exam Trigger Map

- Binary data → byte streams; text → character streams.
- try-with-resources is about reliable cleanup, not faster I/O.
- matches() = whole input; find() = search within input.
- LocalDate has no time; LocalDateTime has no zone; ZonedDateTime has zone context.
- transient affects default serialization, not access control.
- Files.walk returns a stream of paths and should be closed appropriately.

FAST SIGNAL	THINK OF
Trap	Binary data → byte streams; text → character streams.
Trap	try-with-resources is about reliable cleanup, not faster I/O.
Trap	matches() = whole input; find() = search within input.
Trap	LocalDate has no time; LocalDateTime has no zone; ZonedDateTime has zone context.
Trap	transient affects default serialization, not access control.
Trap	Files.walk returns a stream of paths and should be closed appropriately.

MCQ Training Bank — 31 Questions

Question styles are intentionally mixed: direct recall, scenario-based, conceptual reasoning, comparison, trap questions, classification, cardinality/output reasoning and performance/architecture reasoning where appropriate.

Q1 [Original Practice]

A program must read a plain text file containing customer names. Which type of stream is generally more suitable for text?

- A) Character stream
- B) Byte stream only
- C) Object lock
- D) Thread pool

✓ **Answer: A) Character stream**

Why: Character-oriented readers are designed for text and character decoding. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q2 [Original Practice]

What is the main benefit of try-with-resources when working with files or JDBC resources?

- A) It automatically closes eligible resources
- B) It makes every exception checked
- C) It stores files in memory
- D) It prevents all runtime errors

✓ **Answer: A) It automatically closes eligible resources**

Why: Resources implementing AutoCloseable are closed automatically when the try block finishes. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q3 [Original Practice]

Which Java class is appropriate for representing a calendar date without a time-of-day or timezone?

- A) LocalDate
- B) LocalTimeZone
- C) FileDate
- D) ThreadDate

✓ **Answer: A) LocalDate**

Why: LocalDate represents a date such as 2026-10-01 without time or timezone information. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q4 [Original Practice]

A program generates a simple text report containing formatted lines. Which class is convenient for writing textual output with print/println methods?

- A) PrintWriter
- B) FileInputStream
- C) ObjectInputStream
- D) HashMap

✓ **Answer: A) PrintWriter**

Why: PrintWriter is convenient for formatted character/text output. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q5 [Original Practice]

A trainee wants to check whether a product code follows a known text pattern. Which Java API is intended for regular-expression matching?

- A) Pattern and Matcher
- B) LocalDate and Period
- C) List and Set
- D) Thread and Runnable only

✓ **Answer: A) Pattern and Matcher**

Why: `java.util.regex.Pattern` and `Matcher` provide regular-expression support. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q6 [Original Practice]

A `List<Integer>` should produce a new list containing only values greater than 100. Which Streams API operation performs the selection?

- A) filter
- B) map only
- C) reduce only
- D) sorted only

✓ **Answer: A) filter**

Why: `filter` keeps stream elements that satisfy a predicate. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q7 [Original Practice]

A program needs to visit files inside a directory tree using modern Java NIO APIs. Which operation is designed to stream paths recursively?

- A) `Files.walk(...)`
- B) `System.gc()`
- C) `Math.round(...)`
- D) `Thread.sleep(...)`

✓ **Answer: A) `Files.walk(...)`**

Why: `Files.walk` creates a stream of paths for recursive directory traversal. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q8 [Original Practice]

A Serializable User object contains a temporary sessionToken that should not be written during default Java serialization. Which keyword can mark that field?

- A) transient
- B) volatile
- C) abstract
- D) extends

✓ **Answer: A) transient**

Why: transient fields are skipped by default Java object serialization. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q9 [Original Practice]

A log line is "ERROR: payment failed". The regex "payment" should detect that word anywhere inside the line. Which Matcher method is the better fit?

- A) matches(), because it requires the whole line to match
- B) find(), because it searches for a matching subsequence
- C) replaceAll(), because it returns Boolean
- D) group(), before any match attempt

✓ **Answer: B) find(), because it searches for a matching subsequence**

Why: find() searches for a matching subsequence, whereas matches() attempts to match the entire input. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q10 [Direct]

Which Java I/O family is suited to raw image bytes?

- A) FileReader
- B) FileInputStream
- C) PrintWriter only
- D) BufferedReader only

✓ **Answer: C) PrintWriter only**

Why: Byte streams such as FileInputStream handle raw binary data. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q11 [Direct]

Which API is intended for regular expressions?

- A) Pattern and Matcher
- B) LocalDate and Period
- C) HashMap and Set
- D) Thread and Runnable

✓ **Answer: B) LocalDate and Period**

Why: java.util.regex.Pattern and Matcher provide regex compilation and matching. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q12 [Direct]

Which type represents a date without time or timezone?

- A) LocalDate
- B) LocalDateTime
- C) ZonedDateTime
- D) Instant only

✓ **Answer: B) LocalDateTime**

Why: LocalDate represents a calendar date without time-of-day or zone. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q13 [Direct]

Which Matcher method attempts to match the entire input?

- A) find()
- B) matches()
- C) group()
- D) split()

✓ **Answer: C) group()**

Why: matches() requires the pattern to match the whole input. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q14 [Direct]

Which class is convenient for textual output using print/println?

- A) PrintWriter
- B) FileInputStream
- C) ObjectInputStream
- D) Pattern

✓ **Answer: B) FileInputStream**

Why: PrintWriter provides convenient character/text output methods. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q15 [Conceptual]

Why is try-with-resources preferred for files and JDBC resources?

- A) It automatically closes AutoCloseable resources
- B) It disables exceptions
- C) It converts bytes to SQL
- D) It prevents all runtime errors

✓ **Answer: B) It disables exceptions**

Why: Managed resources are closed automatically when the try statement completes. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q16 [Conceptual]

A regex must find the word "payment" anywhere in a log line. Which method fits?

- A) matches()
- B) find()
- C) close()
- D) walk()

✓ **Answer: C) close()**

Why: find() searches for a matching subsequence within the input. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q17 [Conceptual]

A report needs a date and time but no timezone semantics. Which type is most direct?

- A) LocalDate
- B) LocalDateTime
- C) ZonedDateTime
- D) Pattern

✓ **Answer: C) ZonedDateTime**

Why: LocalDateTime combines date and local time without a zone. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q18 [Conceptual]

A scheduler must represent a meeting with its timezone because the same instant is viewed in different zones. Which type is appropriate?

- A) LocalDate
- B) LocalDateTime
- C) ZonedDateTime
- D) String only

✓ **Answer: D) String only**

Why: ZonedDateTime carries date, time, and timezone information. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q19 [Conceptual]

What is the main conceptual difference between Java byte streams and character streams?

- A) Bytes are raw binary-oriented data; characters handle text with character decoding
- B) Characters are always faster
- C) Byte streams can only read databases
- D) Character streams cannot write files

✓ **Answer: B) Characters are always faster**

Why: Byte streams are appropriate for raw bytes; character streams are designed for text. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q20 [Scenario]

A program reads a large directory tree and needs a stream of Path objects for traversal. Which API is designed for this?

- A) Files.walk()
- B) Math.round()
- C) Thread.sleep()
- D) System.gc()

✓ **Answer: B) Math.round()**

Why: Files.walk() produces a stream of paths through a directory tree. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q21 [Scenario]

A service reads a database result set and a file stream in the same method. Which resource-management pattern best reduces leaks?

- A) try-with-resources for eligible resources
- B) Never close either resource
- C) Call System.gc()
- D) Store resources in a Set

✓ **Answer: B) Never close either resource**

Why: try-with-resources reliably closes resources implementing AutoCloseable. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q22 [Scenario]

A product code must be exactly three uppercase letters followed by four digits. Which regex matching choice is appropriate for validating the entire code?

- A) find() only
- B) matches() with the full pattern
- C) group() before matching
- D) Files.walk()

✓ **Answer: C) group() before matching**

Why: Validation of the whole string is the matches() use case. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q23 [Scenario]

A log contains "ERROR: payment failed" and the requirement is to detect whether "payment" occurs anywhere. What should be used?

- A) find()
- B) matches() with pattern payment
- C) LocalDate
- D) PrintWriter

✓ **Answer: B) matches() with pattern payment**

Why: find() locates a matching subsequence within the full log line. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q24 [Scenario]

A temporary sessionToken in a Serializable object should not be included in default serialization. Which modifier is relevant?

- A) transient
- B) final
- C) volatile
- D) synchronized

✓ **Answer: B) final**

Why: transient excludes the field from default Java serialization. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q25 [Scenario]

A program must output formatted text lines to a report file. Which choice is convenient?

- A) PrintWriter
- B) FileInputStream
- C) Matcher
- D) LocalDate

✓ **Answer: B) FileInputStream**

Why: PrintWriter supports convenient text output such as println and printf. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q26 [Conceptual]

Which statement about serialVersionUID is most accurate at this assessment level?

- A) It identifies serialization-version compatibility for a Serializable class
- B) It is a database primary key
- C) It is a thread ID
- D) It stores a file path

✓ **Answer: B) It is a database primary key**

Why: serialVersionUID participates in serialization compatibility checking. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q27 [Scenario]

A program must copy JPEG bytes without interpreting them as characters. Which family is appropriate?

- A) Character streams
- B) Byte streams
- C) Pattern/Matcher
- D) LocalDate

✓ **Answer: B) Byte streams**

Why: Raw binary data is handled with byte streams such as FileInputStream/FileOutputStream. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q28 [Regex trap]

An ID must be exactly three uppercase letters followed by three digits. Which Matcher operation best fits whole-value validation?

- A) find()
- B) matches()
- C) groupCount()
- D) replaceAll()

✓ **Answer: B) matches()**

Why: matches() attempts to match the entire input against the pattern. find() searches for a matching subsequence. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property before selecting the option.

Q29 [Scenario]

Which type is most appropriate for an appointment represented as date + time + geographic time-zone context?

- A) LocalDate
- B) LocalDateTime
- C) ZonedDateTime
- D) String only

✓ **Answer: C) ZonedDateTime**

Why: ZonedDateTime carries date, time and zone information. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q30 [Conceptual]

Why is try-with-resources especially valuable for a file or JDBC connection?

- A) It removes the need for exceptions
- B) It reliably closes managed resources
- C) It converts bytes to characters
- D) It makes all code asynchronous

✓ **Answer: B) It reliably closes managed resources**

Why: try-with-resources closes AutoCloseable resources automatically, reducing leaks even when exceptions occur. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q31 [Trap]

Which statement about transient is correct?

- A) It makes a field private
- B) It excludes the field from default serialization
- C) It makes the field final
- D) It forces database persistence

✓ **Answer: B) It excludes the field from default serialization**

Why: transient tells default Java serialization not to include that field in the serialized state. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: compare the tempting alternatives against the exact definition.

6. Scala Basics & Data Structures

Easy difficulty • 3 official MCQs • 3 official marks • Expanded practice below

What this section is really testing

The handbook frames Scala basics as the syntax literacy required to read and write Spark-oriented code. Focus on immutability, collections, tuples, arrays, pattern matching and guards.

Deep Concept Notes

val vs var

val creates a binding that cannot be reassigned; var permits reassignment. val does not automatically make the referenced object deeply immutable. Scala style generally favors val because fewer reassignments make code easier to reason about.

Immutable List and ::

Scala List is immutable. An expression such as `1 :: xs` returns a new list with 1 prepended; it does not modify xs. The `::` operator is therefore a strong clue that a new list is being constructed rather than an existing list being mutated.

Array

Scala Array is indexed and mutable. Elements can be updated by index. This contrasts with the immutable List used throughout functional-style examples.

Tuples

A tuple groups a fixed number of values that may have different types, such as `(101, "Chennai", 2500.0)`. Values can be accessed positionally or destructured. Tuples are useful for small grouped results rather than representing large domain models.

Type inference

Scala can infer many types from expressions and literals. `val score = 95` is inferred as an `Int` in the usual context. Explicit types can still improve API clarity. Exam questions generally test recognition rather than compiler internals.

Pattern matching

`match/case` selects behavior based on the structure or value of an expression. It is more expressive than a simple equality chain because patterns can deconstruct values. A guard adds an additional boolean condition to a case, written with `if`.

Lambdas and higher-order functions

A function literal such as `x => x * 2` can be passed to collection operations. A higher-order function is one that accepts or returns functions. The exam focus is reading the function's role rather than advanced type theory.

Exam Trigger Map

- `val` prevents reassignment of the binding; it does not guarantee deep immutability of an object.
- `1 :: xs` creates a new List; it does not mutate xs.
- Array is mutable; List is immutable.
- Tuple elements can have different types.
- A guard adds a condition to a pattern case.

FAST SIGNAL	THINK OF
Trap	val prevents reassignment of the binding; it does not guarantee deep immutability of an object.
Trap	1 :: xs creates a new List; it does not mutate xs.
Trap	Array is mutable; List is immutable.
Trap	Tuple elements can have different types.
Trap	A guard adds a condition to a pattern case.

MCQ Training Bank — 31 Questions

Question styles are intentionally mixed: direct recall, scenario-based, conceptual reasoning, comparison, trap questions, classification, cardinality/output reasoning and performance/architecture reasoning where appropriate.

Q1 [Original Practice]

Which Scala keyword creates a reference that cannot be reassigned after initialization?

- A) val
- B) var
- C) def
- D) new

✓ **Answer: A) val**

Why: val creates an immutable binding; var creates a reassignable variable. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q2 [Original Practice]

What is the typical behavior of Scala immutable List operations such as map?

- A) They modify the original list in place
- B) They return a new collection value
- C) They always return Unit
- D) They require a database

✓ **Answer: B) They return a new collection value**

Why: Immutable collection operations derive new values instead of mutating the original list. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q3 [Original Practice]

A Scala program needs to return a customer ID and customer name together without defining a new class. Which simple structure can hold both values?

- A) Tuple
- B) Thread
- C) Exception
- D) RDD action

✓ **Answer: A) Tuple**

Why: A tuple groups a fixed number of values, potentially of different types. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q4 [Original Practice]

Scala sees `val total = 125.50`. What does type inference mean in this statement?

- A) The programmer must write the type later
- B) Scala determines an appropriate type from the assigned value
- C) The value has no type
- D) The variable becomes a String automatically

✓ **Answer: B) Scala determines an appropriate type from the assigned value**

Why: Scala can infer the static type from the expression assigned to the `val`. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q5 [Original Practice]

A status code can be NEW, PAID, or CANCELLED and each value needs a different result. Which Scala feature provides clear case-based branching?

- A) match with case clauses
- B) Only while loops
- C) JDBC commit
- D) RDD cache

✓ **Answer: A) match with case clauses**

Why: Pattern matching with match/case is well suited to branching based on values or shapes. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q6 [Original Practice]

A List(2,4,6) should become List(4,8,12) without changing the original list. Which operation is most suitable?

- A) map
- B) filter
- C) foreach only
- D) dropWhile only

✓ **Answer: A) map**

Why: map transforms each element and returns a new collection. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q7 [Original Practice]

A Scala program needs an indexed collection whose elements can be updated by position. Which basic collection is appropriate?

- A) Array
- B) Immutable Tuple only
- C) Option
- D) Try

✓ **Answer: A) Array**

Why: Scala Array supports indexed access and mutable element updates. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q8 [Original Practice]

Which expression is a valid simple Scala lambda that doubles an integer x?

- A) `x => x * 2`
- B) `x = x * 2` only
- C) `double(x)` class
- D) `case object x * 2`

✓ **Answer: A) `x => x * 2`**

Why: `x => x * 2` is an anonymous function taking x and returning `x*2`. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q9 [Original Practice]

A match case should classify only positive amounts above 1000 as "High". Which form best expresses this additional condition?

- A) case x if x > 1000 => "High"
- B) case x = 1000 => "High"
- C) if case x then 1000
- D) case => x > 1000 only

✓ **Answer: A) case x if x > 1000 => "High"**

Why: A guard adds an if condition to a case pattern and is evaluated when the pattern matches. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q10 [Direct]

Which keyword creates a binding that cannot be reassigned?

- A) val
- B) var
- C) def
- D) new

✓ **Answer: B) var**

Why: val creates a non-reassignable binding. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q11 [Direct]

Which keyword permits reassignment?

- A) val
- B) var
- C) case
- D) object

✓ **Answer: C) case**

Why: var creates a reassignable variable binding. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q12 [Direct]

Which Scala structure groups a fixed number of values that may have different types?

- A) Tuple
- B) Thread
- C) RDD action
- D) Pattern

✓ **Answer: B) Thread**

Why: A tuple groups a fixed number of values and can contain different types. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q13 [Direct]

What does Scala type inference do?

- A) Determines a static type from the expression/context
- B) Removes all types
- C) Converts every value to String
- D) Makes every variable mutable

✓ **Answer: B) Removes all types**

Why: Scala can infer the static type of many expressions without an explicit annotation. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q14 [Direct]

What is the typical mutability property of Scala List?

- A) Immutable
- B) Always mutable
- C) Thread-local only
- D) Database-backed

✓ **Answer: B) Always mutable**

Why: Scala List is an immutable collection in the standard collection model. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q15 [Conceptual]

What does Array provide that is especially useful for indexed updates?

- A) Mutable indexed elements
- B) Guaranteed immutability
- C) Key-value lookup
- D) Pattern matching only

✓ **Answer: B) Guaranteed immutability**

Why: Arrays support indexed access and mutable element updates. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q16 [Conceptual]

What does the wildcard case `_` usually mean in a match expression?

- A) A catch-all/default case
- B) An array index
- C) A tuple field
- D) A database key

✓ **Answer: B) An array index**

Why: The underscore pattern can match any remaining value when used as a catch-all case. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q17 [Conceptual]

What is a guard in Scala pattern matching?

- A) An additional Boolean condition on a case
- B) A database lock
- C) A collection mutation
- D) A JVM bytecode file

✓ **Answer: B) A database lock**

Why: A guard uses syntax such as `case x if condition` to refine a pattern. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q18 [Conceptual]

Which statement about val is most accurate?

- A) The binding cannot be reassigned, though a referenced mutable object can still change
- B) Everything reachable from it becomes immutable
- C) It is the same as var
- D) It is a database constant

✓ **Answer: B) Everything reachable from it becomes immutable**

Why: val fixes the reference/binding, not necessarily the deep mutability of the referenced object. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q19 [Scenario]

A function receives status codes NEW, PAID, and CANCELLED and must choose different results. Which Scala feature is a natural fit?

- A) match with cases
- B) Array mutation only
- C) JDBC rollback
- D) Files.walk

✓ **Answer: B) Array mutation only**

Why: Pattern matching provides clear case-based branching. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q20 [Scenario]

A trainee writes val total = 125.50 without a type annotation. What is happening?

- A) Scala infers an appropriate static numeric type
- B) Scala stores it as untyped memory
- C) It becomes a String
- D) It becomes var automatically

✓ **Answer: B) Scala stores it as untyped memory**

Why: The compiler infers a static type from the assigned literal. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q21 [Scenario]

A function must return customerId and customerName together without a new class. Which structure is concise?

- A) Tuple
- B) HashSet
- C) Thread
- D) Exception only

✓ **Answer: B) HashSet**

Why: A tuple is designed for grouping a fixed set of values. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q22 [Scenario]

A program must update an element at index 2 in a collection. Which choice is more directly suited?

- A) Array
- B) Immutable List
- C) Option
- D) Either

✓ **Answer: B) Immutable List**

Why: Array supports indexed element update; immutable List operations produce new values. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q23 [Scenario]

A pattern should match positive integers differently from other integers. Which construct can add the condition?

- A) Guard such as case x if x > 0
- B) tuple projection only
- C) val reassignment
- D) Source.close()

✓ **Answer: B) tuple projection only**

Why: A guard adds a Boolean condition to a pattern case. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q24 [Conceptual]

A developer wants a value that cannot be rebound but wants to mutate the object it references when the object itself is mutable. Which concept is consistent?

- A) val binding plus mutable referenced object
- B) var only
- C) final class only
- D) None

✓ **Answer: B) var only**

Why: val prevents reassignment of the reference, not all possible mutations inside the referenced object. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q25 [Scenario]

Which operation on an immutable List normally returns a new collection instead of changing the original?

- A) map
- B) direct element assignment
- C) field mutation
- D) database commit

✓ **Answer: B) direct element assignment**

Why: Immutable collection transformations derive new collection values. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q26 [Direct]

Which expression is a Scala function literal?

- A) `x => x * 2`
- B) `class x`
- C) `new x()`
- D) `import x`

✓ **Answer: B) class x**

Why: `x => x * 2` is an anonymous function/lambda expression. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q27 [Direct]

After `val x = 10`, which operation is conceptually disallowed?

- A) Reading x
- B) Using x in an expression
- C) Reassigning x to another value
- D) Printing x

✓ **Answer: C) Reassigning x to another value**

Why: `val` creates a binding that cannot be reassigned. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q28 [Scenario]

If `xs` is an immutable List, what does `1 :: xs` conceptually do?

- A) Mutates `xs` in place
- B) Returns a new List with 1 at the front
- C) Converts `xs` to Array
- D) Removes the first element

✓ **Answer: B) Returns a new List with 1 at the front**

Why: The List is immutable; `::` constructs a new list with the new head. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q29 [Scenario]

Which structure is the clearest fit when indexed mutation is required?

- A) Immutable List
- B) Array
- C) Option
- D) Tuple

✓ **Answer: B) Array**

Why: Scala Array supports indexed access and element updates. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q30 [Direct]

What does a guard add to a match case?

- A) A database connection
- B) An additional boolean condition
- C) A new collection
- D) A type conversion

✓ **Answer: B) An additional boolean condition**

Why: A guard adds an if condition that must also be true for the case to match. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q31 [Conceptual]

Which statement about tuples is correct?

- A) Every element must have the same type
- B) A tuple groups a fixed number of values and can contain different types
- C) Tuples are always mutable
- D) Tuples are database rows only

✓ **Answer: B) A tuple groups a fixed number of values and can contain different types**

Why: Tuples group a fixed number of values and may contain different types. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

7. Scala RDDs

Easy difficulty • 4 official MCQs • 4 official marks • Expanded practice below

What this section is really testing

RDD questions are heavily about execution behavior: transformation vs action, lazy evaluation, lineage, narrow vs wide operations, pair RDD aggregation, joins, persistence and driver-memory risks. The handbook expects conceptual recognition rather than scheduler internals.

Deep Concept Notes

Creating RDDs

`sc.parallelize` creates an RDD from a driver-resident local collection and is mainly useful for small examples/testing. `sc.textFile` creates an RDD from an external text source. The scenario tells you which creation path fits.

Transformations

`map`, `filter`, `flatMap` and `reduceByKey` are transformations. They define new RDDs but are lazy: Spark records what should be done and does not fully execute the lineage until an action is called.

Actions

`count`, `collect`, `first/take`, `reduce` and `save` operations are actions. They trigger execution and either return results to the driver or write results externally. `collect` is especially important because it transfers all elements to the driver.

Lazy evaluation

Spark delays transformations until an action is needed. This allows Spark to build an execution plan and avoid executing transformations whose results are never consumed. MCQs often ask what happens before `count/collect/save`: the transformations are defined but not fully executed.

Lineage and fault tolerance

Lineage is the dependency chain of transformations used to derive an RDD. If a partition is lost, Spark can recompute it from lineage. Lineage is not the same thing as checkpoint data: checkpointing materializes recovery state to durable storage.

Narrow vs wide

Narrow transformations such as typical `map/filter` can compute an output partition from a limited set of input partitions without broad redistribution. Wide operations such as many `group/repartition/aggregation` patterns require shuffle or data movement. The word “shuffle” is the main exam clue for a wide dependency.

Pair RDDs

Pair RDDs contain key/value tuples. `reduceByKey` aggregates values for the same key and performs distributed aggregation rather than collecting all values to the driver. `join` combines records from two pair RDDs by key.

Persistence

`cache/persist` keeps computed partitions for reuse. It is useful when an expensive intermediate RDD is reused by multiple actions or downstream computations. Caching everything can waste memory; persistence should be justified by reuse.

mapPartitions

`mapPartitions` runs a function once per partition and supplies an iterator. It can reduce repeated per-record setup when expensive initialization can be shared across records in a partition. It is not the default replacement for ordinary `map` logic.

Exam Trigger Map

- Transformations are lazy; actions trigger execution.
- collect() returns all data to the driver and can cause driver OOM.
- Lineage supports recomputation; checkpoint materializes recovery state.
- map/filter are typical narrow transformations.
- reduceByKey is distributed per-key aggregation.
- Persist when reuse justifies the storage cost.

FAST SIGNAL	THINK OF
Trap	Transformations are lazy; actions trigger execution.
Trap	collect() returns all data to the driver and can cause driver OOM.
Trap	Lineage supports recomputation; checkpoint materializes recovery state.
Trap	map/filter are typical narrow transformations.
Trap	reduceByKey is distributed per-key aggregation.
Trap	Persist when reuse justifies the storage cost.

MCQ Training Bank — 32 Questions

Question styles are intentionally mixed: direct recall, scenario-based, conceptual reasoning, comparison, trap questions, classification, cardinality/output reasoning and performance/architecture reasoning where appropriate.

Q1 [Original Practice]

Which of the following is an RDD transformation that returns a new RDD?

- A) map
- B) count
- C) collect
- D) first

✓ **Answer: A) map**

Why: map is a lazy transformation that returns another RDD. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q2 [Original Practice]

Which RDD operation is an action that returns the number of elements?

- A) count
- B) filter
- C) map
- D) flatMap

✓ **Answer: A) count**

Why: count triggers computation and returns a value to the driver. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q3 [Original Practice]

What does lazy evaluation mean for Spark RDD transformations?

- A) They run immediately when written
- B) Spark records transformations and executes them when an action requires results
- C) They never execute
- D) Only the driver can use them

✓ **Answer: B) Spark records transformations and executes them when an action requires results**

Why: Transformations build a lineage plan and are evaluated when an action is invoked. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q4 [Original Practice]

An RDD partition is lost after an executor failure. How can Spark often rebuild it?

- A) Using the recorded lineage of transformations
- B) Only from a manual backup file
- C) By restarting the operating system
- D) By converting it to a Java List

✓ **Answer: A) Using the recorded lineage of transformations**

Why: RDD lineage records how partitions were derived, enabling recomputation when needed. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q5 [Original Practice]

A pair RDD contains (product, amount) records. Which operation is commonly used to combine amounts for the same product efficiently?

- A) reduceByKey
- B) collect before aggregation
- C) mapPartitions only
- D) repartition to one partition first

✓ **Answer: A) reduceByKey**

Why: reduceByKey combines values by key in distributed processing. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q6 [Original Practice]

A small Scala collection already exists on the driver for a training example. Which Spark method can distribute it as an RDD?

- A) sc.parallelize(collection)
- B) sc.collect(collection)
- C) df.show(collection)
- D) spark.stop(collection)

✓ **Answer: A) sc.parallelize(collection)**

Why: SparkContext.parallelize distributes an existing local collection into an RDD. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q7 [Original Practice]

The same expensive RDD is reused in several actions during one application. Which Spark feature can avoid recomputing it each time?

- A) persist/cache
- B) collect then delete the RDD
- C) Rename the RDD
- D) Convert every element to String

✓ **Answer: A) persist/cache**

Why: Caching or persistence stores computed partitions for reuse. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q8 [Original Practice]

Why is map usually described as a narrow transformation?

- A) Each output partition can be computed from a limited input partition without a full data shuffle
- B) It always reduces partitions to one
- C) It broadcasts every record
- D) It runs only on the driver

✓ **Answer: A) Each output partition can be computed from a limited input partition without a full data shuffle**

Why: map has a narrow dependency because each output partition depends on a small number, usually one, of input partitions. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q9 [Original Practice]

A 500 GB RDD is processed successfully, but the driver fails when collect() is called. What is the main reason?

- A) collect() attempts to bring all elements to the driver
- B) collect() deletes executor memory
- C) collect() disables Spark
- D) collect() changes every value to null

✓ **Answer: A) collect() attempts to bring all elements to the driver**

Why: collect() transfers the full result to the driver, which can exceed driver memory for large datasets. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q10 [Direct]

Which call creates an RDD from an in-memory Scala collection?

- A) `sc.parallelize(data)`
- B) `sc.textFile(data)`
- C) `df.collect(data)`
- D) `spark.write(data)`

✓ **Answer: B) `sc.textFile(data)`**

Why: parallelize distributes a local collection into an RDD. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q11 [Direct]

Which call reads text files into an RDD?

- A) `sc.textFile(path)`
- B) `sc.parallelize(path)`
- C) `df.explain(path)`
- D) `sc.commit(path)`

✓ **Answer: B) `sc.parallelize(path)`**

Why: `textFile` creates an RDD from text-file input. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q12 [Direct]

Which is a Spark transformation?

- A) `map`
- B) `count`
- C) `collect`
- D) `first`

✓ **Answer: B) `count`**

Why: `map` defines a new RDD lazily. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q13 [Direct]

Which is a Spark action?

- A) `filter`
- B) `map`
- C) `count`
- D) `flatMap`

✓ **Answer: D) `flatMap`**

Why: `count` triggers execution and returns a result. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q14 [Direct]

What does lazy evaluation mean for RDD transformations?

- A) Transformations define computation but are not executed until an action requires a result
- B) Transformations always execute immediately
- C) Spark never executes transformations
- D) Only the driver can run transformations

✓ **Answer: B) Transformations always execute immediately**

Why: Spark builds a lineage/plan and executes it when an action is invoked. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q15 [Conceptual]

What is lineage?

- A) The dependency chain describing how an RDD was derived
- B) A database connection string
- C) A JVM stack trace only
- D) A Parquet schema

✓ **Answer: B) A database connection string**

Why: Lineage records the transformation dependencies needed for recomputation. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q16 [Conceptual]

Why can collect() be dangerous on a large RDD?

- A) It brings all records to the driver
- B) It deletes the RDD
- C) It disables partitions
- D) It always causes deadlock

✓ **Answer: B) It deletes the RDD**

Why: collect transfers the entire result to driver memory, which can exhaust it. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q17 [Conceptual]

When is cache/persist most useful?

- A) When an expensive dataset will be reused across actions
- B) When a dataset is used once only
- C) When you need to change schema
- D) When you need JDBC transactions

✓ **Answer: B) When a dataset is used once only**

Why: Persistence avoids recomputing reused data from lineage. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q18 [Conceptual]

Which operation is conceptually a wide dependency because it may require data redistribution?

- A) reduceByKey
- B) map
- C) filter
- D) mapValues only in a narrow case

✓ **Answer: B) map**

Why: Per-key aggregation generally requires records with the same key to meet, causing shuffle/redistribution. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q19 [Scenario]

A developer has a 10-million-row RDD and needs only the first five records for a sanity check. Which is safer?

- A) take(5)
- B) collect()
- C) toLocalIterator over everything
- D) convert to a giant local List first

✓ **Answer: B) collect()**

Why: take retrieves a small sample instead of transferring the whole RDD. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q20 [Scenario]

A pair RDD contains (region, amount) and the requirement is total amount per region. Which operation fits?

- A) reduceByKey(_ + _)
- B) filter(region)
- C) collect()
- D) first()

✓ **Answer: B) filter(region)**

Why: reduceByKey aggregates values sharing the same key. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q21 [Scenario]

An expensive RDD is used by three separate actions. What can reduce repeated recomputation?

- A) cache/persist the RDD
- B) call collect() first
- C) convert it to a local array
- D) call first() only

✓ **Answer: B) call collect() first**

Why: Persisting the reused RDD can avoid recomputing its lineage for later actions. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q22 [Scenario]

An executor loses a partition of a non-persisted RDD. What concept lets Spark rebuild it?

- A) Lineage and recomputation
- B) JDBC commit
- C) regex find
- D) CopyOnWriteArrayList

✓ **Answer: B) JDBC commit**

Why: Spark can recompute lost partitions from their transformation lineage. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q23 [Scenario]

A per-record operation repeatedly creates the same expensive client object. Which RDD transformation can allow setup once per partition?

- A) mapPartitions
- B) collect
- C) count
- D) first

✓ **Answer: B) collect**

Why: mapPartitions runs a function over an iterator for each partition, enabling shared setup per partition. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q24 [Direct]

Two pair RDDs must be combined using their common key. Which operation is the natural choice?

- A) join
- B) count
- C) take
- D) filter

✓ **Answer: B) count**

Why: join combines pair RDD records by matching keys. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q25 [Conceptual]

Which mental rule is most useful for identifying an action?

- A) It triggers execution and returns/writes an observable result
- B) It always creates a new RDD
- C) It never runs code
- D) It must be a Scala keyword

✓ **Answer: B) It always creates a new RDD**

Why: Actions force Spark to execute the needed lineage to produce a result or output. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q26 [Conceptual]

Which statement best distinguishes narrow from wide dependencies?

- A) Narrow avoids broad redistribution; wide may require shuffle between partitions
- B) Narrow always runs on driver; wide always runs on one executor
- C) Narrow means immutable; wide means mutable
- D) They are identical

✓ **Answer: B) Narrow always runs on driver; wide always runs on one executor**

Why: Wide dependencies require data movement across partitions; narrow dependencies can be computed from limited upstream partitions. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q27 [Execution reasoning]

A pipeline has only map and filter, followed by no action. What is the most accurate description?

- A) All data is immediately recomputed
- B) The transformations are recorded lazily and execution waits for an action
- C) The RDD is automatically written to disk
- D) collect runs implicitly

✓ **Answer: B) The transformations are recorded lazily and execution waits for an action**

Why: RDD transformations are lazy. An action such as count, collect or save is needed to trigger execution. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: separate lazy definition from actual execution.

Q28 [Scenario]

A 500-GB RDD is followed by collect(). What is the main risk?

- A) The RDD becomes immutable
- B) All records are brought to the driver, risking driver memory exhaustion
- C) Spark automatically compresses the driver RAM
- D) collect converts it to a DataFrame

✓ **Answer: B) All records are brought to the driver, risking driver memory exhaustion**

Why: collect returns the entire result to the driver. Large results can exceed driver memory. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q29 [Scenario]

A costly intermediate RDD is used by three actions. Which concept most directly addresses repeated recomputation?

- A) cache/persist
- B) collect
- C) first only
- D) repartition to one

✓ **Answer: A) cache/persist**

Why: Persistence stores computed partitions for reuse, avoiding repeated recomputation when the same RDD is reused. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q30 [Conceptual]

Which statement best describes lineage?

- A) A list of all driver variables
- B) The dependency chain used to derive an RDD
- C) A copy of every record on disk
- D) A JDBC connection pool

✓ **Answer: B) The dependency chain used to derive an RDD**

Why: Lineage records how the RDD was derived and allows Spark to recompute lost partitions. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q31 [Scenario]

Why is reduceByKey a natural choice for per-key totals?

- A) It centralizes all records at the driver
- B) It aggregates values associated with each key in distributed processing
- C) It only works on unkeyed data
- D) It disables shuffles in every case

✓ **Answer: B) It aggregates values associated with each key in distributed processing**

Why: reduceByKey is designed for key/value RDD aggregation and performs distributed aggregation rather than requiring all values at the driver. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q32 [Classification]

Which operation is most clearly an action?

- A) filter
- B) map
- C) count
- D) flatMap

✓ **Answer: C) count**

Why: count is an action that triggers execution and returns a result. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property before selecting the option.

8. Scala File I/O

Easy difficulty • 2 official MCQs • 2 official marks • Expanded practice below

What this section is really testing

The handbook's Scala I/O section is small but important because the exam uses conceptual questions about resource management and error-handling types. Know `Source`, `getLines/mkString`, `close()`, `Option`, `Try` and `Either`.

Deep Concept Notes

`scala.io.Source`

`Source` is a basic Scala utility for reading text from files or URLs. `fromFile` creates a source; `getLines` supports line-oriented processing and `mkString` combines the content into one `String`.

Resource closure

A `Source` owns an underlying resource such as a file handle. It should be closed after use, ideally reliably even when an exception occurs. MCQs may show `Source.fromFile(...).mkString` and ask what is missing: the important issue is resource closure.

`mkString` vs `getLines`

`mkString` is convenient when you want the entire text as one `String`, especially for small inputs. `getLines` is appropriate for line-by-line processing and can avoid building one giant `String` unnecessarily.

`Option`

`Option` models an optional value: `Some(value)` when present and `None` when absent. It is appropriate when absence is a normal outcome rather than an exceptional failure. `getOrElse` provides a fallback without forcing an unsafe direct extraction.

`Try`

`Try` represents a computation that may succeed or fail with an exception: `Success(value)` or `Failure(exception)`. It is appropriate when exception-producing operations need to be represented and composed as values.

`Either`

`Either` can carry one of two alternatives. In the handbook's validation framing, `Right` represents success and `Left` can carry explicit error information. It is useful when you need more detail about why a validation failed than `Option` provides.

Exam Trigger Map

- Source must be closed.
- `mkString` reads the whole content into one `String`; it is not ideal for huge files.
- `Option` = expected presence/absence.
- `Try` = success/failure of an exception-producing computation.
- `Either` = success plus explicit error information in the other branch.

FAST SIGNAL	THINK OF
Trap	Source must be closed.
Trap	<code>mkString</code> reads the whole content into one <code>String</code> ; it is not ideal for huge files.
Trap	<code>Option</code> = expected presence/absence.

FAST SIGNAL	THINK OF
Trap	Try = success/failure of an exception-producing computation.
Trap	Either = success plus explicit error information in the other branch.

MCQ Training Bank — 30 Questions

Question styles are intentionally mixed: direct recall, scenario-based, conceptual reasoning, comparison, trap questions, classification, cardinality/output reasoning and performance/architecture reasoning where appropriate.

Q1 [Original Practice]

Which Scala utility is commonly used in simple examples to read text from a file?

- A) scala.io.Source
- B) scala.collection.Set
- C) SparkSession.stop
- D) ThreadLocal

✓ **Answer: A) scala.io.Source**

Why: scala.io.Source provides simple methods for reading text input. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q2 [Original Practice]

What does Option help represent in Scala?

- A) A value that may be present or absent
- B) Only a mutable list
- C) A network socket
- D) A thread lock

✓ **Answer: A) A value that may be present or absent**

Why: Option models optional values as Some(value) or None. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q3 [Original Practice]

A file-reading operation may throw an exception. Which Scala type can capture success or failure as a value?

- A) Try
- B) Tuple2 only
- C) Array
- D) RDD partition

✓ **Answer: A) Try**

Why: Try represents a computation that may result in Success or Failure. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q4 [Original Practice]

A Source is opened to read a text file. What should the program ensure after reading is complete?

- A) The Source is closed
- B) The JVM is always restarted
- C) The file is converted into an RDD
- D) The path is stored in a Set

✓ **Answer: A) The Source is closed**

Why: Closing the Source releases the underlying file resource. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q5 [Original Practice]

A validation function should return either an error message or a valid Order. Which Scala type naturally models two alternative result types?

- A) Either
- B) Array only
- C) Unit
- D) Thread

✓ **Answer: A) Either**

Why: Either represents one of two alternatives, commonly Left(error) or Right(value). The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q6 [Original Practice]

A trainee opens `Source.fromFile(path)`. Which method is commonly used to iterate through file lines?

- A) `getLines()`
- B) `collect()`
- C) `reduceByKey()`
- D) `commit()`

✓ **Answer: A) `getLines()`**

Why: `getLines()` returns an iterator over the text lines from the Source. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q7 [Original Practice]

A `Try[Int]` should return 0 when parsing fails. Which concept is designed to provide a fallback for a failed `Try`?

- A) `recover` or `getOrElse`
- B) RDD checkpoint only
- C) Thread interrupt
- D) `Pattern.compile` only

✓ **Answer: A) `recover` or `getOrElse`**

Why: `Try` supports recovery/fallback operations such as `recover` and `getOrElse`. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q8 [Original Practice]

A Map lookup returns `Option[String]`. Which approach safely provides "Unknown" when no value exists?

- A) `option.getOrElse("Unknown")`
- B) `option.get` always
- C) `option.close()`
- D) `option.collectAsMap()`

✓ **Answer: A) `option.getOrElse("Unknown")`**

Why: `getOrElse` returns the contained value for `Some` or the supplied default for `None`. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q9 [Original Practice]

A program must open a file path and then parse an integer from its contents, and either step may fail. Why can wrapping the operations in Try be useful?

- A) It lets success/failure flow through composable operations instead of requiring unchecked assumptions
- B) It guarantees the file exists
- C) It converts every exception to null
- D) It makes all I/O asynchronous

✓ **Answer: A) It lets success/failure flow through composable operations instead of requiring unchecked assumptions**

Why: Try makes failure explicit as a value and can be composed with map/flatMap while preserving failure information. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q10 [Direct]

Which Scala utility is commonly used to read text files?

- A) scala.io.Source
- B) scala.concurrent.Future only
- C) scala.collection.Map only
- D) java.time.LocalDate

✓ **Answer: B) scala.concurrent.Future only**

Why: scala.io.Source provides convenient text input utilities. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q11 [Direct]

Which Source method provides an iterator over lines?

- A) getLines()
- B) reduceByKey()
- C) collect()
- D) commit()

✓ **Answer: B) reduceByKey()**

Why: getLines() provides line-oriented iteration. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q12 [Direct]

What does mkString conceptually do?

- A) Combines elements/characters into one String
- B) Opens a JDBC transaction
- C) Creates an RDD
- D) Adds elements to a Set

✓ **Answer: B) Opens a JDBC transaction**

Why: mkString combines elements into textual content. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q13 [Direct]

What does Option represent?

- A) A value that may be present or absent
- B) A guaranteed exception
- C) A database connection
- D) A Spark shuffle

✓ **Answer: B) A guaranteed exception**

Why: Option uses Some and None to model optional values. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q14 [Direct]

What does Try represent?

- A) A computation that may succeed or fail with an exception
- B) Only missing values
- C) A collection iterator
- D) A lock

✓ **Answer: B) Only missing values**

Why: Try contains Success(value) or Failure(exception). The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q15 [Conceptual]

What is the common convention for Either?

- A) Left carries error/failure information and Right carries success
- B) Left is always success
- C) Both sides must be exceptions
- D) Either is only a file reader

✓ **Answer: B) Left is always success**

Why: Either is commonly used as Left(error) or Right(success). The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q16 [Conceptual]

When is Option generally more appropriate than Try?

- A) When absence is expected rather than an exception
- B) When you must preserve an exception object
- C) When you need a JDBC connection
- D) When you need a shuffle

✓ **Answer: B) When you must preserve an exception object**

Why: Option models presence/absence; Try models exception-producing computations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q17 [Conceptual]

Why can Either be preferable to Option for validation?

- A) It can carry meaningful error information in Left
- B) It automatically retries JDBC
- C) It guarantees no errors
- D) It makes files immutable

✓ **Answer: B) It automatically retries JDBC**

Why: Either can distinguish different error information instead of collapsing failure to None. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q18 [Conceptual]

What is `getOrElse` used for on `Option` or `Try`-like values?

- A) Supplying a fallback when the expected value is absent/fails
- B) Closing a file
- C) Starting a Spark job
- D) Acquiring a lock

✓ **Answer: B) Closing a file**

Why: `getOrElse` provides a default when no successful/available value is present. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q19 [Scenario]

A file is opened with `Source.fromFile` and read using `getLines`. What should happen after use?

- A) Close the Source
- B) Commit the Source
- C) Cache the Source forever
- D) Convert it to a `HashMap`

✓ **Answer: B) Commit the Source**

Why: Closing releases the underlying file resource. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q20 [Scenario]

A parser may throw `NumberFormatException` and the program wants a value-oriented `Success/Failure` result. Which type fits?

- A) `Try[Int]`
- B) `Option[Int]` only
- C) `Tuple[Int]` only
- D) `Set[Int]`

✓ **Answer: B) `Option[Int]` only**

Why: `Try` captures successful values or thrown exceptions as `Success/Failure`. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q21 [Scenario]

A customer lookup normally returns no customer for some IDs, and that absence is not exceptional. Which type is conceptually best?

- A) Option[Customer]
- B) Try[Customer] only
- C) Either[IOException, Customer] always
- D) Thread

✓ **Answer: B) Try[Customer] only**

Why: Option naturally models expected absence with Some/None. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q22 [Scenario]

A validation routine should return either "invalid amount" or a valid Order. Which type is natural?

- A) Either[String, Order]
- B) Option[Order] only
- C) Array[Order]
- D) Source

✓ **Answer: B) Option[Order] only**

Why: Either can carry an explicit error on the Left and a valid value on the Right. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q23 [Scenario]

A Try[Int] fails and the application wants 0 for a particular exception. Which operation is relevant?

- A) recover or getOrElse
- B) unionByName
- C) reduceByKey
- D) explain

✓ **Answer: B) unionByName**

Why: Try provides fallback operations including recover and getOrElse. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q24 [Scenario]

A small text file must be loaded entirely into one String. Which Source operation is convenient?

- A) mkString
- B) getLines only without consuming
- C) reduceByKey
- D) commit

✓ **Answer: B) getLines only without consuming**

Why: mkString combines the source content into a single String and is convenient for small text. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q25 [Conceptual]

Why should a large file not automatically be read with mkString?

- A) It materializes all text as one String and can consume substantial memory
- B) It cannot read text
- C) It always shuffles data
- D) It closes JDBC connections

✓ **Answer: B) It cannot read text**

Why: mkString is convenient for small content but materializing huge files as one String can be costly. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q26 [Conceptual]

Which mapping is correct?

- A) Option=presence/absence; Try=exception success/failure; Either=success/error information
- B) Option=database; Try=thread; Either=file
- C) Option=shuffle; Try=Parquet; Either=JVM
- D) All three mean exactly the same thing

✓ **Answer: B) Option=database; Try=thread; Either=file**

Why: These three types model different kinds of uncertainty/failure. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q27 [Direct]

Why should a `scala.io.Source` be closed after reading?

- A) To make the text immutable
- B) To release the underlying resource
- C) To trigger a Spark shuffle
- D) To convert Source into Option

✓ **Answer: B) To release the underlying resource**

Why: Closing releases the underlying file/resource handle. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q28 [Scenario]

A missing configuration value is a normal possibility rather than an exceptional failure. Which type fits best?

- A) Option
- B) Try
- C) Thread
- D) Either only

✓ **Answer: A) Option**

Why: Option represents expected presence/absence using Some or None. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q29 [Scenario]

A parsing operation may throw an exception and you want success/failure represented as a value. Which type is most appropriate?

- A) List
- B) Try
- C) Array
- D) Tuple

✓ **Answer: B) Try**

Why: Try represents Success or Failure for exception-producing computations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q30 [Scenario]

A validation API must return either a valid result or a meaningful error message. Which type is especially suitable?

- A) Option
- B) Either
- C) Array
- D) Unit

✓ **Answer: B) Either**

Why: Either can carry explicit error information in one branch while the other branch carries success. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

9. Advanced Spark

Medium difficulty • 2 official MCQs • 4 official marks • Expanded practice below

What this section is really testing

This is a medium-weight, high-value topic because each question is worth two marks in the supplied bifurcation. The handbook focuses on reading execution plans, recognizing Exchange/shuffle, choosing built-ins over opaque UDFs, filtering before expensive custom work, partitioned JDBC reads and Parquet pruning.

Deep Concept Notes

Logical vs physical plans

A logical plan describes what a query means; a physical plan describes how Spark intends to execute it. `df.explain()` is the practical inspection tool. At this level, you do not need to memorize every operator; recognize that explain is the diagnostic window into execution planning.

Exchange and shuffle

Exchange commonly represents redistribution/repartitioning of data. Shuffle is expensive because data moves between partitions/executors. Aggregations, joins and explicit repartitioning can introduce such movement. If a question asks what a performance-sensitive Exchange indicates, think data redistribution.

Built-ins vs UDFs

Built-in Spark SQL/DataFrame functions expose their semantics to Spark and are generally easier for Spark to optimize than opaque custom UDF code. Prefer built-ins when an equivalent exists; use a UDF when the required custom logic is not conveniently expressible with built-ins.

Filter before expensive UDF

If a cheap filter can eliminate many rows before an expensive custom function, the UDF processes fewer rows. This is a general pipeline principle: reduce data before expensive work when doing so does not change semantics.

JDBC partitioned reads

`numPartitions` controls the number of read partitions/concurrent tasks in a partitioned JDBC read. The purpose is to parallelize reading from a database rather than pulling the entire table through one task. Partitioning must be configured around an appropriate partition column/range.

Parquet column pruning

Parquet is columnar. Column pruning means reading only the columns needed by the query instead of scanning every column. This reduces I/O and can improve performance. Do not confuse column pruning with partition pruning.

Partition pruning

Partition pruning skips entire data partitions when the query filter identifies which partition values are relevant. It reduces the amount of data that needs to be scanned. Column pruning removes unnecessary columns; partition pruning removes unnecessary partitions.

Exam Trigger Map

- `explain()` → execution plan inspection.
- Exchange → think redistribution/shuffle.
- Built-in function available → usually prefer it over UDF.
- Filter early when it safely reduces expensive downstream work.

- numPartitions in JDBC → parallel read partitioning.
- Column pruning ≠ partition pruning.

FAST SIGNAL	THINK OF
explain()	execution plan inspection.
Exchange	think redistribution/shuffle.
Built-in function available	usually prefer it over UDF.
Trap	Filter early when it safely reduces expensive downstream work.
numPartitions in JDBC	parallel read partitioning.
Trap	Column pruning ≠ partition pruning.

MCQ Training Bank — 32 Questions

Question styles are intentionally mixed: direct recall, scenario-based, conceptual reasoning, comparison, trap questions, classification, cardinality/output reasoning and performance/architecture reasoning where appropriate.

Q1 [Original Practice]

Which DataFrame method is commonly used to inspect how Spark plans to execute a query?

- A) explain()
- B) showSchemaOnly()
- C) collectPlan()
- D) savePlan()

✓ **Answer: A) explain()**

Why: DataFrame.explain() displays logical/physical planning information at an appropriate level of detail. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q2 [Original Practice]

When a transformation can be expressed using built-in Spark SQL functions, which approach is generally preferred?

- A) Use built-in functions
- B) Always write a custom UDF
- C) Collect data to the driver
- D) Convert everything to text files first

✓ **Answer: A) Use built-in functions**

Why: Built-in functions are visible to Spark optimization and generally avoid unnecessary opaque UDF logic. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q3 [Original Practice]

Why is Parquet often efficient for analytical queries that select only a few columns?

- A) It is columnar and can avoid reading unnecessary columns
- B) It stores every row as an image
- C) It requires collect() first
- D) It runs only in memory

✓ **Answer: A) It is columnar and can avoid reading unnecessary columns**

Why: Parquet is a columnar format, enabling column pruning for analytical workloads. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q4 [Original Practice]

A Spark job needs to read a table from a relational database. Which Spark data source is appropriate?

- A) JDBC
- B) Socket text only
- C) RDD checkpoint
- D) LocalDate

✓ **Answer: A) JDBC**

Why: Spark JDBC connectivity is intended for reading from and writing to relational databases. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q5 [Original Practice]

A trainee wants to know which join operator Spark actually selected after planning. Which plan is most relevant?

- A) Physical plan
- B) Only the source file name
- C) DataFrame schema only
- D) Java heap dump

✓ **Answer: A) Physical plan**

Why: The physical plan shows the concrete physical operators selected for execution. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q6 [Original Practice]

A Parquet dataset is partitioned by event_date. A query filters event_date to one day. What optimization can avoid scanning unrelated date folders?

- A) Partition pruning
- B) Autoboxing
- C) Thread synchronization
- D) Object serialization

✓ **Answer: A) Partition pruning**

Why: Partition pruning uses partition filters to skip irrelevant partitions. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q7 [Original Practice]

A Parquet table has 150 columns, but a report selects only 5. Which optimization reduces unnecessary column I/O?

- A) Column pruning
- B) Deadlock detection
- C) JDBC rollback
- D) Scala pattern guards

✓ **Answer: A) Column pruning**

Why: Column pruning lets Spark read only required columns from columnar storage when possible. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q8 [Original Practice]

A very large JDBC table is currently read through a single input partition. At a conceptual level, what can improve source-side parallel reading when a suitable numeric key exists?

- A) Configure partitioned JDBC reading using a partition column and multiple partitions
- B) Call collect() immediately
- C) Increase only the number of DataFrame columns
- D) Write the table to a Java array first

✓ **Answer: A) Configure partitioned JDBC reading using a partition column and multiple partitions**

Why: Partitioned JDBC reads can issue multiple range queries when an appropriate partitioning column and bounds are provided. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q9 [Original Practice]

Two implementations return the same cleaned customer name. One uses built-in trim/upper functions; the other uses a custom UDF. Why is the built-in expression usually easier for Spark to optimize?

- A) Its transformation semantics are visible to Spark SQL planning
- B) It always uses one partition
- C) It automatically caches all results
- D) It executes only on the driver

✓ **Answer: A) Its transformation semantics are visible to Spark SQL planning**

Why: Built-in expressions expose structured semantics to Spark, whereas custom UDF logic is more opaque to the optimizer. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q10 [Direct]

What does `DataFrame.explain()` help inspect?

- A) Logical/physical execution planning information
- B) JVM source code
- C) JDBC passwords
- D) File permissions only

✓ **Answer: B) JVM source code**

Why: explain displays planning/execution information useful for diagnosing Spark behavior. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q11 [Direct]

What does Exchange commonly indicate in a physical plan?

- A) Data redistribution/shuffle
- B) Java compilation
- C) Regex matching
- D) File closure

✓ **Answer: B) Java compilation**

Why: Exchange commonly represents repartitioning/data movement between stages. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q12 [Direct]

When a built-in Spark SQL function can perform the required transformation, what is generally preferred?

- A) Built-in function
- B) Custom UDF by default
- C) collect first
- D) Local loop on driver

✓ **Answer: B) Custom UDF by default**

Why: Built-ins expose semantics to Spark and are generally easier to optimize. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q13 [Direct]

What does numPartitions control conceptually in a partitioned JDBC read?

- A) Number of parallel read partitions/tasks
- B) Number of DataFrame columns
- C) JVM heap size
- D) Number of regex groups

✓ **Answer: B) Number of DataFrame columns**

Why: numPartitions controls parallel partitioned JDBC reading and concurrent source-side work. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q14 [Direct]

What is Parquet?

- A) A columnar storage format
- B) A Java thread type
- C) A JDBC transaction
- D) A Scala error type

✓ **Answer: B) A Java thread type**

Why: Parquet stores data in a column-oriented format suited to analytics. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q15 [Conceptual]

Why can filter-before-UDF improve performance?

- A) It reduces the number of rows on which the expensive UDF runs
- B) It makes the UDF more opaque
- C) It disables partitioning
- D) It sends data to the driver

✓ **Answer: B) It makes the UDF more opaque**

Why: A selective filter can reduce downstream custom computation. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q16 [Conceptual]

What is column pruning?

- A) Reading only required columns from a columnar source when possible
- B) Deleting table columns permanently
- C) Reducing JDBC connections
- D) Removing partitions

✓ **Answer: B) Deleting table columns permanently**

Why: Column pruning reduces unnecessary column I/O. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q17 [Conceptual]

What is partition pruning?

- A) Skipping irrelevant storage partitions using partition filters
- B) Reducing Java stack frames
- C) Deleting DataFrame columns
- D) Closing JDBC connections

✓ **Answer: B) Reducing Java stack frames**

Why: Partition pruning avoids scanning unrelated partition directories/files. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q18 [Conceptual]

Why should JDBC numPartitions not be increased without regard to the database?

- A) Excessive parallel reads can overload the source database
- B) It makes all rows local
- C) It disables JDBC
- D) It forces JSON output

✓ **Answer: B) It makes all rows local**

Why: Parallelism must be balanced against source database capacity. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q19 [Scenario]

A query is slow and contains a large aggregation. What tool can reveal whether a shuffle/Exchange is involved?

- A) `df.explain(true)`
- B) `LocalDate.now()`
- C) `Pattern.compile()`
- D) `Iterator.remove()`

✓ **Answer: B) LocalDate.now()**

Why: The execution plan can reveal Exchange and physical operators used by the aggregation. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q20 [Scenario]

A cleaning operation can be expressed as trim and upper. One implementation uses built-ins; another uses a UDF. Which is generally preferable?

- A) Built-ins
- B) UDF always
- C) collect first
- D) convert to RDD solely for trimming

✓ **Answer: B) UDF always**

Why: Built-ins preserve semantics visible to Spark optimization. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q21 [Scenario]

Only 5% of records need an expensive custom calculation. What ordering is usually sensible when logically valid?

- A) Selective filter first, then UDF
- B) UDF first, then filter
- C) collect first
- D) write JSON first

✓ **Answer: B) UDF first, then filter**

Why: Filtering first reduces the input to the expensive custom computation. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q22 [Scenario]

A Parquet table is partitioned by event_date and the query filters one date. What can reduce directory/file scanning?

- A) Partition pruning
- B) Column pruning only
- C) Autoboxing
- D) Deadlock prevention

✓ **Answer: B) Column pruning only**

Why: A filter on the partition column can skip unrelated partitions. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q23 [Scenario]

A Parquet table has 200 columns and the report needs five. Which optimization targets unnecessary column reads?

- A) Column pruning
- B) Partition pruning
- C) JDBC rollback
- D) Lock ordering

✓ **Answer: B) Partition pruning**

Why: Column pruning reduces I/O by reading only needed columns where supported. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q24 [Scenario]

A very large database table has a suitable numeric ID column. Which configuration concept can enable parallel JDBC reads?

- A) partitionColumn with bounds and numPartitions
- B) collect()
- C) cache only
- D) unionByName

✓ **Answer: B) collect()**

Why: Partitioned JDBC reads use a partition column and bounds to divide the source read. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q25 [Conceptual]

What is a practical distinction between logical and physical plans?

- A) Logical describes what the query means; physical describes how Spark will execute it
- B) Logical is always faster
- C) Physical contains Java source code
- D) They are unrelated to Spark SQL

✓ **Answer: B) Logical is always faster**

Why: Spark first reasons about the logical computation and then selects physical operators. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q26 [Conceptual]

Which statement about JSON versus Parquet matches the preparation guidance?

- A) JSON is flexible for ingestion; Parquet is columnar and efficient for analytics
- B) JSON is always faster for analytics
- C) Parquet is only for Java objects
- D) Both formats have identical storage behavior

✓ **Answer: B) JSON is always faster for analytics**

Why: The handbook emphasizes choosing formats based on stage/workload: flexible JSON ingestion and efficient columnar Parquet analytics. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q27 [Scenario]

A developer wants to know whether an aggregation introduces data redistribution. Which tool is most directly useful?

- A) `df.explain()`
- B) `println` of one row only
- C) `LocalDate`
- D) `Source.close()`

✓ **Answer: A) `df.explain()`**

Why: `explain` exposes the logical/physical execution plan, where operators such as `Exchange` can reveal redistribution. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q28 [Plan reasoning]

An `Exchange` appears in a physical plan. What should an exam candidate infer first?

- A) Only local computation occurred
- B) Data redistribution/repartitioning is involved
- C) The query is a Java exception
- D) The source is necessarily a socket

✓ **Answer: B) Data redistribution/repartitioning is involved**

Why: `Exchange` commonly represents repartitioning/data movement, often associated with shuffle. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: map the physical-plan term to data movement.

Q29 [Performance]

A built-in Spark expression and a UDF can both implement the same simple transformation. Which is generally preferred?

- A) UDF because it is always faster
- B) Built-in expression because Spark can understand and optimize its semantics
- C) Neither can be used
- D) Socket source

✓ **Answer: B) Built-in expression because Spark can understand and optimize its semantics**

Why: Built-ins expose semantics to Spark and are generally more optimizer-friendly than opaque UDFs. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: look for the option that reduces unnecessary work or data movement.

Q30 [Scenario]

A UDF is expensive and a cheap filter can safely remove 90% of rows first. What is the best reasoning?

- A) Apply the UDF to all rows first
- B) Filter first so fewer rows reach the expensive UDF
- C) collect before the UDF
- D) Convert the data to a socket

✓ **Answer: B) Filter first so fewer rows reach the expensive UDF**

Why: Reducing the data before expensive work lowers the number of expensive evaluations when semantics permit. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q31 [JDBC reasoning]

A JDBC table is large and can be split using an appropriate numeric partition column. What does numPartitions primarily help control?

- A) Number of JDBC read partitions/tasks
- B) Number of columns in a row
- C) Number of Java classes
- D) Number of regex groups

✓ **Answer: A) Number of JDBC read partitions/tasks**

Why: numPartitions controls parallelism for the partitioned JDBC read. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property before selecting the option.

Q32 [Comparison]

A query selects only three columns from a wide Parquet dataset containing 80 columns. Which optimization directly describes reading only the needed columns?

- A) Partition pruning
- B) Column pruning
- C) Deadlock prevention
- D) Watermarking

✓ **Answer: B) Column pruning**

Why: Column pruning avoids reading columns that the query does not need. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: eliminate options that swap the two concepts.

10. Scala Transformations

Medium difficulty • 2 official MCQs • 4 official marks • Expanded practice below

What this section is really testing

The transformation section is medium difficulty and directly tests whether you can reason about collection pipelines. The most important distinction is the output shape: map preserves one output per input; filter keeps or discards whole elements; flatMap can produce multiple/zero outputs per input and then flattens them.

Deep Concept Notes

map

map transforms every input element into one output element. If the input has N elements and the function succeeds for each, the output has N elements. Use it when the requirement says “convert/transform each element.”

filter

filter keeps elements for which a predicate returns true. Its function has the conceptual type $A \Rightarrow \text{Boolean}$. Output cardinality is less than or equal to input cardinality.

flatMap

flatMap combines mapping and flattening. One input can produce zero, one or multiple outputs, and nested collections are flattened one level. It is the natural choice when each input record can expand into several output records.

Pipeline order

Operations execute in the order written. Filtering before an expensive transformation can reduce the number of elements that reach the expensive step. When a question asks for the most efficient logically equivalent pipeline, look for early selective filters.

Pure functions

A pure function's result depends only on its explicit inputs and it avoids observable side effects. Such functions are easier to test and reason about. In functional Spark/Scala questions, purity is usually the safer conceptual choice.

Higher-order functions

Collection methods such as map/filter/flatMap accept functions as arguments. Recognizing the expected function shape helps decode MCQs quickly: map accepts an element-to-result function; filter accepts a predicate; flatMap accepts an element-to-collection function.

Exam Trigger Map

- map → one output per input.
- filter → predicate $A \Rightarrow \text{Boolean}$.
- flatMap → zero/many outputs + flatten.
- Filtering before expensive work is often preferable when semantics allow it.
- Pure functions avoid hidden state/side effects.

FAST SIGNAL	THINK OF
map	one output per input.
filter	predicate $A \Rightarrow \text{Boolean}$.

FAST SIGNAL	THINK OF
flatMap	zero/many outputs + flatten.
Trap	Filtering before expensive work is often preferable when semantics allow it.
Trap	Pure functions avoid hidden state/side effects.

MCQ Training Bank — 31 Questions

Question styles are intentionally mixed: direct recall, scenario-based, conceptual reasoning, comparison, trap questions, classification, cardinality/output reasoning and performance/architecture reasoning where appropriate.

Q1 [Original Practice]

Which collection operation transforms every element and returns a new collection?

- A) map
- B) filter
- C) exists
- D) foreach only

✓ **Answer: A) map**

Why: map applies a function to each element and returns transformed results. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q2 [Original Practice]

Which Scala collection operation keeps only elements that satisfy a Boolean condition?

- A) filter
- B) map
- C) flatten only
- D) zip only

✓ **Answer: A) filter**

Why: filter retains elements for which the predicate returns true. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q3 [Original Practice]

When each input element can produce zero or more output elements and the final result should be flat, which operation is especially useful?

- A) flatMap
- B) head
- C) size
- D) reverse only

✓ **Answer: A) flatMap**

Why: flatMap maps each element to a collection-like result and flattens the outputs. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q4 [Original Practice]

A list of transactions must first remove invalid records and then normalize the remaining records. Which chain best matches the required order?

- A) transactions.filter(valid).map(normalize)
- B) transactions.map(normalize).foreach(valid)
- C) transactions.foreach(valid).filter(normalize)
- D) transactions.reduce(valid).map(normalize)

✓ **Answer: A) transactions.filter(valid).map(normalize)**

Why: filter first ensures normalize runs only for records that pass validation. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q5 [Original Practice]

A reusable select function should accept the rule to apply from the caller. Which parameter type represents a predicate for elements of type A?

- A) A => Boolean
- B) String only
- C) Boolean => A always
- D) Unit => Unit only

✓ **Answer: A) A => Boolean**

Why: A => Boolean is a function type that accepts an A and returns a Boolean decision. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q6 [Original Practice]

A tax function should return the same result for the same transaction and tax rate every time. Which design best supports this?

- A) Pass required values as parameters and return the result without changing shared state
- B) Read a changing global tax rate
- C) Update a shared mutable variable during calculation
- D) Write to a global buffer before returning

✓ **Answer: A) Pass required values as parameters and return the result without changing shared state**

Why: Pure-style functions avoid hidden mutable state and base results on explicit inputs. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q7 [Original Practice]

A method receives an immutable List and must remove invalid values. What should it do?

- A) Return a new filtered list
- B) Cast and mutate the original list
- C) Store it globally and modify it later
- D) Return Unit and change external state

✓ **Answer: A) Return a new filtered list**

Why: Immutable collection operations derive a new collection and leave the original unchanged. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q8 [Original Practice]

Only active customers should contribute their verified contacts, and each customer can have many contacts. Which chain best fits?

- A) `customers.filter(_.active).flatMap(_.contacts).filter(_.verified)`
- B) `customers.map(_.contacts)` only
- C) `customers.foreach(_.contacts).map(_.verified)`
- D) `customers.head.contacts` only

✓ **Answer: A) `customers.filter(_.active).flatMap(_.contacts).filter(_.verified)`**

Why: filter selects active customers, flatMap flattens their contact lists, and the final filter keeps verified contacts. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q9 [Original Practice]

A pipeline must validate records, convert valid ones into objects, and emit zero or more results per object without mutable buffers. Which sequence best expresses the flow?

- A) filter followed by map followed by flatMap
- B) foreach followed by global ArrayBuffer updates
- C) collect everything first, then mutate a shared list
- D) Use only while loops with shared state

✓ **Answer: A) filter followed by map followed by flatMap**

Why: filter/map/flatMap compose naturally while returning new values and avoiding shared mutable accumulation. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q10 [Direct]

What does map generally do to a collection?

- A) Transforms each element and returns one result per input
- B) Removes all elements
- C) Groups by database key only
- D) Opens a file

✓ **Answer: B) Removes all elements**

Why: map applies a function to each element and produces corresponding outputs. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q11 [Direct]

What does filter do?

- A) Keeps elements whose predicate is true
- B) Transforms every value into a pair
- C) Flattens nested collections only
- D) Sorts by default

✓ **Answer: B) Transforms every value into a pair**

Why: filter selects elements satisfying a Boolean predicate. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q12 [Direct]

What is the key behavior of flatMap?

- A) Map to collection-like outputs and flatten them
- B) Keep only one element
- C) Always sort values
- D) Convert all values to String

✓ **Answer: B) Keep only one element**

Why: flatMap combines mapping with flattening, allowing zero or more outputs per input. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q13 [Direct]

Which function type is a predicate for values of type A?

- A) $A \Rightarrow \text{Boolean}$
- B) $\text{Boolean} \Rightarrow A$
- C) $A \Rightarrow \text{Unit}$ only
- D) $\text{String} \Rightarrow \text{String}$ only

✓ **Answer: B) $\text{Boolean} \Rightarrow A$**

Why: A predicate accepts A and returns a Boolean decision. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q14 [Direct]

What is a pure function?

- A) Its result depends on explicit inputs and it avoids hidden side effects
- B) It must modify global state
- C) It always performs I/O
- D) It must be asynchronous

✓ **Answer: B) It must modify global state**

Why: Pure-style functions are deterministic with respect to explicit inputs and avoid hidden mutation/I/O. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q15 [Conceptual]

If `List(1,2,3).map(x => x*2)` is evaluated, what is the shape of the result?

- A) Three transformed values
- B) Six nested lists
- C) One Boolean
- D) Empty list always

✓ **Answer: B) Six nested lists**

Why: `map` preserves the number of elements for ordinary one-to-one transformations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q16 [Conceptual]

If `List(1,2,3).filter(_ > 1)` is evaluated, what determines which elements remain?

- A) Predicate result
- B) Return type of the collection
- C) JVM heap size
- D) Database commit

✓ **Answer: B) Return type of the collection**

Why: `filter` retains elements for which its predicate evaluates to true. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q17 [Conceptual]

If each word maps to a list of tokens, which operation is appropriate to obtain one flat token list?

- A) `flatMap`
- B) `map` only
- C) `filter` only
- D) `head`

✓ **Answer: B) map only**

Why: `flatMap` maps each item to multiple outputs and flattens the nested results. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q18 [Conceptual]

Why is filter often placed before an expensive map transformation?

- A) It can reduce the number of elements needing the expensive work
- B) It forces a shuffle
- C) It makes the list mutable
- D) It disables type checking

✓ **Answer: B) It forces a shuffle**

Why: Early selective filtering can reduce downstream computation. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q19 [Scenario]

A transaction list contains invalid records that should never reach normalization logic. Which pipeline is best?

- A) filter(valid).map(normalize)
- B) map(normalize).filter(valid)
- C) foreach(valid).reduce(normalize)
- D) flatMap(normalize).head

✓ **Answer: B) map(normalize).filter(valid)**

Why: Filtering first ensures invalid records are removed before normalization. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q20 [Scenario]

Each input sentence can produce several words and the final result must be one list of words. Which operation fits?

- A) flatMap
- B) filter
- C) map to nested lists and stop
- D) exists

✓ **Answer: B) filter**

Why: flatMap handles one-to-many mapping followed by flattening. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q21 [Scenario]

A tax function reads a mutable global rate that can change during execution. Why is it not a good pure function?

- A) Its result depends on hidden shared state
- B) It uses a lambda
- C) It accepts parameters
- D) It returns a number

✓ **Answer: B) It uses a lambda**

Why: A pure function should derive its result from explicit inputs rather than hidden mutable state. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q22 [Scenario]

A reusable selection utility accepts a rule from its caller. Which signature is appropriate?

- A) A => Boolean
- B) A => A only
- C) Boolean => Boolean only
- D) Unit

✓ **Answer: B) A => A only**

Why: A predicate is represented by a function from A to Boolean. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q23 [Scenario]

A list has 1,000 records but only 20 satisfy a cheap validity rule. Which order can reduce expensive work?

- A) filter first, then expensive map
- B) expensive map first, then filter
- C) collect first, then map
- D) sort first regardless

✓ **Answer: B) expensive map first, then filter**

Why: Selective filtering first reduces the number of records reaching the expensive stage. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q24 [Conceptual]

What happens to the original immutable List after map?

- A) It remains unchanged; a new collection value is produced
- B) It is overwritten in place
- C) It becomes mutable
- D) It is deleted

✓ **Answer: B) It is overwritten in place**

Why: Immutable transformations return new values rather than mutating the original list. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q25 [Direct]

Which transformation is most naturally described as "one input can produce zero or more outputs, then flatten"?

- A) flatMap
- B) filter
- C) map
- D) foreach

✓ **Answer: B) filter**

Why: That is the defining conceptual pattern of flatMap. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q26 [Conceptual]

A function returns the same output whenever the same explicit arguments are supplied and performs no I/O. Which property does it illustrate?

- A) Purity/determinism
- B) Deadlock
- C) Schema inference
- D) Serialization

✓ **Answer: B) Deadlock**

Why: Dependence only on explicit inputs with no hidden side effects is the core pure-function idea. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q27 [Cardinality]

A list of 10 records is passed through map with a successful one-to-one function. How many output elements should you expect?

- A) Exactly 0
- B) At most 5
- C) 10
- D) Unlimited

✓ **Answer: C) 10**

Why: map produces one output for each input element, so a successful one-to-one mapping preserves cardinality. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property before selecting the option.

Q28 [Direct]

A filter predicate returns false for 30% of records. What does filter do?

- A) Transforms every record into two records
- B) Removes records for which the predicate is false
- C) Sorts records
- D) Groups by key automatically

✓ **Answer: B) Removes records for which the predicate is false**

Why: filter keeps only elements whose predicate is true. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q29 [Scenario]

Each customer record can produce zero, one or many tags, and the final result should be one flat sequence of tags. Which operation is the natural fit?

- A) map
- B) filter
- C) flatMap
- D) count

✓ **Answer: C) flatMap**

Why: flatMap supports zero/many outputs per input and flattens the nested results. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q30 [Conceptual]

Why can pure functions make transformation pipelines easier to test?

- A) They depend on hidden global state
- B) Their output depends on explicit inputs and avoids hidden side effects
- C) They always run in parallel
- D) They mutate input collections

✓ **Answer: B) Their output depends on explicit inputs and avoids hidden side effects**

Why: Pure functions are deterministic with respect to their explicit inputs and avoid hidden side effects. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q31 [Optimization reasoning]

A pipeline can either filter 95% of records before an expensive transformation or after it, with identical final semantics. Which is usually preferable?

- A) Filter first
- B) Filter last
- C) Never filter
- D) collect first

✓ **Answer: A) Filter first**

Why: Filtering early reduces the number of elements that must undergo the expensive later transformation. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: reduce data/work before expensive operations when semantics allow.

11. Scala DataFrames

Medium difficulty • 2 official MCQs • 4 official marks • Expanded practice below

What this section is really testing

DataFrame questions test structured data reasoning: schemas, explicit schema vs inference, expressions, unionByName, missing columns and why DataFrames are optimizer-friendly. The exam is conceptual rather than deep Catalyst internals.

Deep Concept Notes

Why DataFrames

A DataFrame represents structured data with named columns and a schema. Spark can reason about these expressions and optimize them. This makes DataFrames a natural choice for SQL-style analytics compared with untyped low-level RDD processing.

Explicit schema

An explicit schema tells Spark the expected column names and data types. It improves predictability and avoids depending on inference from sample data. It is especially useful when data types must be controlled consistently.

inferSchema

Schema inference asks Spark to inspect input data and infer types. It is convenient, but inference is not a substitute for validation: malformed values or inconsistent data can still create problems depending on the reader and options.

Expressions and built-ins

DataFrame transformations are commonly expressed through column expressions and built-in functions. These expressions expose structure to Spark's optimizer. This is one reason built-ins are generally preferable to opaque UDFs when equivalent.

unionByName

unionByName aligns columns by their names rather than relying only on positional order. allowMissingColumns can permit missing columns and fill them with nulls. The key exam clue is "schemas have the same logical columns but different order" → unionByName.

DataFrameWriter

DataFrameWriter is used to write DataFrame results to external storage formats/sinks. MCQs may test recognition of the writer API versus reading APIs or transformation methods.

Malformed input and DecimalType

A declared DecimalType expresses the intended schema type; it does not magically make every source string a valid decimal. Bad input can still fail parsing or be handled as malformed/null depending on the reader configuration.

Exam Trigger Map

- DataFrame = structured named columns + schema.
- Explicit schema improves predictability.
- inferSchema guesses; it does not validate every source value.
- unionByName matches columns by name.
- allowMissingColumns can fill absent columns with null.
- DecimalType does not guarantee valid source strings.

FAST SIGNAL	THINK OF
Trap	DataFrame = structured named columns + schema.
Trap	Explicit schema improves predictability.
Trap	inferSchema guesses; it does not validate every source value.
Trap	unionByName matches columns by name.
Trap	allowMissingColumns can fill absent columns with null.
Trap	DecimalType does not guarantee valid source strings.

MCQ Training Bank — 31 Questions

Question styles are intentionally mixed: direct recall, scenario-based, conceptual reasoning, comparison, trap questions, classification, cardinality/output reasoning and performance/architecture reasoning where appropriate.

Q1 [Original Practice]

A CSV contract already defines exact column names and types. What should Spark use when creating the DataFrame for consistent interpretation?

- A) An explicit schema
- B) A random schema each run
- C) Only a local List
- D) No column names

✓ **Answer: A) An explicit schema**

Why: An explicit schema enforces the intended structure consistently across files. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q2 [Original Practice]

For structured rows with named columns, filters, joins, and aggregations, which Spark abstraction is usually more suitable?

- A) DataFrame
- B) Plain local Array only
- C) Driver-side HashMap only
- D) Thread

✓ **Answer: A) DataFrame**

Why: DataFrames provide named columns and structured relational expressions optimized by Spark SQL. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q3 [Original Practice]

What does inferSchema do when reading supported data such as CSV?

- A) Spark examines data to guess column data types
- B) It encrypts the file
- C) It caches all rows permanently
- D) It creates a JDBC connection

✓ **Answer: A) Spark examines data to guess column data types**

Why: Schema inference samples/examines data to infer likely types rather than requiring every type to be supplied explicitly. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q4 [Original Practice]

Two DataFrames contain the same fields in different column orders. Which operation can append rows by matching column names rather than positions?

- A) unionByName
- B) join only
- C) collect
- D) groupByKey on an RDD

✓ **Answer: A) unionByName**

Why: unionByName aligns columns by name, avoiding position-based mismatches. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q5 [Original Practice]

df2 is missing one column that exists in df1. Which option allows unionByName to fill the missing column with null?

- A) allowMissingColumns = true
- B) inferSchema = false
- C) header = true
- D) cache = true

✓ **Answer: A) allowMissingColumns = true**

Why: unionByName(..., allowMissingColumns=true) permits missing fields and fills them with null. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q6 [Original Practice]

Two versions produce the same output. One uses DataFrame column expressions; the other converts rows to arbitrary objects and opaque functions. Which version exposes more relational information to Spark?

- A) The DataFrame expression version
- B) The local object version
- C) Both always expose identical semantics
- D) The version that calls collect() first

✓ **Answer: A) The DataFrame expression version**

Why: Structured DataFrame expressions expose column and relational semantics to Spark SQL planning. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q7 [Original Practice]

A large DataFrame result must remain distributed and be written to shared storage. Which approach is appropriate?

- A) Use DataFrameWriter to write the distributed result
- B) collect() everything and use one local file writer
- C) Convert it to one Scala List first
- D) Print every row on the driver

✓ **Answer: A) Use DataFrameWriter to write the distributed result**

Why: Distributed DataFrame writes avoid bringing the entire result to the driver. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q8 [Original Practice]

A schema declares amount as Decimal, but a source record contains invalid text. Which statement is most accurate?

- A) The schema states the intended type, but malformed input still depends on parsing and validation behavior
- B) The schema guarantees all business values are valid
- C) Spark always repairs every malformed value
- D) Invalid text automatically becomes a valid decimal

✓ **Answer: A) The schema states the intended type, but malformed input still depends on parsing and validation behavior**

Why: Schema declaration defines expected structure; it does not guarantee that all incoming source values are valid business data. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q9 [Original Practice]

A workload mainly performs projections, filters, joins, and aggregations on structured order records. The team wants Spark to inspect these operations for optimization. Why is a DataFrame a stronger fit than an RDD of opaque domain objects?

- A) DataFrames express named-column relational operations that Spark SQL can analyze
- B) RDDs cannot be distributed
- C) DataFrames always avoid shuffles
- D) RDDs execute only on the driver

✓ **Answer: A) DataFrames express named-column relational operations that Spark SQL can analyze**

Why: DataFrame expressions expose structured semantics to Spark SQL/Catalyst, while arbitrary RDD functions are more opaque. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q10 [Direct]

Which Spark abstraction is generally preferred for structured tabular analytics?

- A) DataFrame
- B) Raw byte array
- C) Thread
- D) LocalDate

✓ **Answer: B) Raw byte array**

Why: DataFrames provide schema-aware, named-column relational operations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q11 [Direct]

What does inferSchema do?

- A) Spark examines input and infers likely data types
- B) It encrypts data
- C) It caches forever
- D) It creates JDBC transactions

✓ **Answer: B) It encrypts data**

Why: Schema inference lets Spark infer types from the source data. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q12 [Direct]

What is the main benefit of an explicit schema?

- A) Predictable names/types and controlled ingestion
- B) It guarantees all source values are valid
- C) It removes all nulls
- D) It prevents every shuffle

✓ **Answer: B) It guarantees all source values are valid**

Why: Explicit schemas make expected structure and types predictable. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q13 [Direct]

Which method unions DataFrames by matching column names?

- A) unionByName
- B) join only
- C) collect
- D) cache

✓ **Answer: B) join only**

Why: unionByName aligns columns by name rather than only position. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q14 [Direct]

Which option permits unionByName to tolerate missing columns by filling them with null?

- A) allowMissingColumns=true
- B) inferSchema=false
- C) header=false
- D) cache=true

✓ **Answer: B) inferSchema=false**

Why: allowMissingColumns permits missing fields and supplies null for them. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q15 [Conceptual]

Why can DataFrame expressions be easier for Spark to optimize than arbitrary object functions?

- A) They expose named-column relational semantics to the optimizer
- B) They always avoid shuffles
- C) They run only on the driver
- D) They disable schema

✓ **Answer: B) They always avoid shuffles**

Why: Structured expressions preserve semantics Spark SQL can analyze and optimize. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q16 [Conceptual]

Does declaring DecimalType guarantee every malformed source string becomes a valid decimal?

- A) No
- B) Yes, always
- C) Only if collect is called
- D) Only on the driver

✓ **Answer: B) Yes, always**

Why: A schema expresses expected types; malformed input can still fail or be treated according to reader behavior. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q17 [Conceptual]

What does DataFrameWriter conceptually provide?

- A) Distributed output of DataFrame results to storage
- B) A JVM compiler
- C) A regex matcher
- D) A lock manager

✓ **Answer: B) A JVM compiler**

Why: DataFrameWriter writes distributed DataFrame results to supported destinations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q18 [Conceptual]

Why might a DataFrame be preferred over an RDD of opaque domain objects for joins and aggregations?

- A) Spark can analyze named-column relational expressions
- B) RDDs cannot be distributed
- C) DataFrames never shuffle
- D) DataFrames always fit in driver memory

✓ **Answer: B) RDDs cannot be distributed**

Why: Structured semantics give Spark more optimization opportunities. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q19 [Scenario]

Two DataFrames have columns id,name,amount and amount,id,name. What operation can append them correctly by name?

- A) unionByName
- B) collect
- C) first
- D) reduceByKey

✓ **Answer: B) collect**

Why: unionByName matches corresponding columns by name. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q20 [Scenario]

DataFrame df2 lacks a column present in df1, and the requirement is to union while inserting null for that field. What should be enabled?

- A) allowMissingColumns=true
- B) inferSchema=true only
- C) cache=true
- D) checkpoint=true

✓ **Answer: B) inferSchema=true only**

Why: allowMissingColumns allows the missing field to be filled with null. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q21 [Scenario]

A production pipeline has known fields and types across many daily files. Which ingestion choice gives the strongest type predictability?

- A) Explicit schema
- B) Always inferSchema
- C) collect first
- D) Convert everything to String manually

✓ **Answer: B) Always inferSchema**

Why: An explicit schema controls expected structure and types consistently. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q22 [Scenario]

A source column is declared DecimalType but one record contains "abc". What should the engineer conclude?

- A) The schema does not guarantee valid source data; parsing/reader behavior still matters
- B) abc must become a valid decimal automatically
- C) Spark will always silently fix it
- D) DecimalType disables validation

✓ **Answer: B) abc must become a valid decimal automatically**

Why: Schema declaration and source-data validity are separate concerns. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q23 [Scenario]

A large DataFrame result must be written to shared storage without bringing all rows to the driver. Which approach fits?

- A) DataFrameWriter
- B) collect then local writer
- C) convert to one local List
- D) print rows on driver

✓ **Answer: B) collect then local writer**

Why: Distributed writing keeps the result distributed and avoids a driver-side materialization. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q24 [Scenario]

A query uses `filter(col("amount") > 1000).groupBy("region").sum("amount")`. Why is this form useful to Spark?

- A) It exposes structured column and relational semantics
- B) It forces every row to the driver
- C) It hides all operations in an opaque UDF
- D) It disables planning

✓ **Answer: B) It forces every row to the driver**

Why: Built-in DataFrame expressions are visible to Spark SQL planning. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q25 [Conceptual]

Which statement best describes a DataFrame schema?

- A) It describes column names, types and related structural metadata
- B) It is only a file path
- C) It is a thread lock
- D) It is a cache block

✓ **Answer: B) It is only a file path**

Why: Schema describes the structure of structured data. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q26 [Conceptual]

What is the key distinction between `unionByName` and a position-based row append?

- A) `unionByName` aligns fields by column name
- B) `unionByName` always joins by primary key
- C) `unionByName` performs JDBC reads
- D) `unionByName` only works on RDDs

✓ **Answer: B) `unionByName` always joins by primary key**

Why: `unionByName` is specifically designed to align matching columns by name. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q27 [Conceptual]

Why can Spark optimize a DataFrame query more effectively than opaque arbitrary object processing?

- A) DataFrames have no schema
- B) Named columns and expressions expose structure/semantics to Spark
- C) DataFrames always fit in driver memory
- D) DataFrames disable query planning

✓ **Answer: B) Named columns and expressions expose structure/semantics to Spark**

Why: Structured schemas and column expressions allow Spark to reason about the computation and optimize it. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q28 [Scenario]

Two DataFrames contain the same logical columns but in different orders. Which operation is designed to align by column name?

- A) unionByName
- B) collect
- C) cache only
- D) reduceByKey

✓ **Answer: A) unionByName**

Why: unionByName aligns columns by name rather than relying purely on positional order. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q29 [Direct]

One DataFrame lacks a column present in the other. Which feature can allow the union while filling the missing column with nulls?

- A) allowMissingColumns
- B) inferSchema=false
- C) watermark
- D) Iterator.remove

✓ **Answer: A) allowMissingColumns**

Why: allowMissingColumns supports unions where columns are absent on one side, filling missing values with nulls. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q30 [Trap]

Which is the strongest reason to prefer an explicit schema for important ingestion pipelines?

- A) It guarantees all source strings are valid
- B) It makes expected names and types predictable
- C) It removes all malformed data
- D) It forces every column to String

✓ **Answer: B) It makes expected names and types predictable**

Why: An explicit schema makes the expected structure and data types predictable; it does not validate arbitrary input values. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: compare the tempting alternatives against the exact definition.

Q31 [Scenario]

A source has inconsistent values in a decimal column. Declaring DecimalType means:

- A) Every source value is automatically valid
- B) The intended schema type is decimal, but parsing can still fail or produce malformed/null results depending on options
- C) The source becomes Parquet automatically
- D) The query becomes streaming

✓ **Answer: B) The intended schema type is decimal, but parsing can still fail or produce malformed/null results depending on options**

Why: Schema declaration expresses the target structure; it cannot make invalid source text valid. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

12. Spark Streaming & Fault Tolerance

Difficult difficulty • 2 official MCQs • 4 official marks • Expanded practice below

What this section is really testing

This is the difficult Spark section in the supplied blueprint. Focus on Structured Streaming, DStreams as the legacy model, checkpoints, windows, event time, watermarks, triggers and source durability. The exam emphasizes conceptual recovery and state-management reasoning.

Deep Concept Notes

Structured Streaming vs DStreams

Structured Streaming is the modern DataFrame-oriented streaming API. DStreams are the older micro-batch abstraction built around RDDs. For new DataFrame-style streaming applications, Structured Streaming is the expected answer.

Checkpointing

Checkpoint data stores progress/state needed to recover a streaming query after failure. It must be stored durably so it survives process or driver failure. Checkpointing is about recovery state/progress, not merely caching.

Windows

A window groups events over a time range. Window length describes how much event-time history belongs to a window; slide interval describes how often the window advances/produces overlapping results. A smaller slide than window length creates overlapping windows.

Watermarks

A watermark expresses how long the engine is willing to wait for late event-time data before considering older state eligible for cleanup. It helps bound state and manage late events. It is not the same thing as a trigger interval.

Trigger

A trigger controls when Spark processes/updates available data. A watermark controls late-event/state-management policy. Questions that contrast “how often processing runs” with “how late data is accepted/state retained” are distinguishing trigger from watermark.

Sources

File sources can provide durable arriving data. A socket source is mainly useful for demos/testing because it is ephemeral and not designed as a durable replayable production source. The source's durability affects recovery reasoning.

Fault tolerance model

Spark's recovery story depends on durable checkpoint information and a source from which required data can be replayed/re-read. A recovery mechanism is only as strong as the durability of the metadata and the source data it depends on.

Exam Trigger Map

- Structured Streaming = modern DataFrame streaming.
- DStream = legacy RDD streaming model.
- Checkpoint = durable recovery/progress/state.
- Watermark = late data + state cleanup; trigger = processing schedule.
- Window length $\neq$ slide interval.

- Socket source is typically demo/testing, not durable production ingestion.

FAST SIGNAL	THINK OF
Trap	Structured Streaming = modern DataFrame streaming.
Trap	DStream = legacy RDD streaming model.
Trap	Checkpoint = durable recovery/progress/state.
Trap	Watermark = late data + state cleanup; trigger = processing schedule.
Trap	Window length ≠ slide interval.
Trap	Socket source is typically demo/testing, not durable production ingestion.

MCQ Training Bank — 32 Questions

Question styles are intentionally mixed: direct recall, scenario-based, conceptual reasoning, comparison, trap questions, classification, cardinality/output reasoning and performance/architecture reasoning where appropriate.

Q1 [Original Practice]

Which Spark API provides DataFrame/Dataset-style processing for streaming data?

- A) Structured Streaming
- B) Java Serialization
- C) RDD collect
- D) JDBC only

✓ **Answer: A) Structured Streaming**

Why: Structured Streaming uses the structured DataFrame/Dataset model for streaming computations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q2 [Original Practice]

Which statement best describes DStreams in modern Spark training?

- A) They are the older RDD-based streaming abstraction
- B) They are Parquet files
- C) They are Java interfaces
- D) They are database indexes

✓ **Answer: A) They are the older RDD-based streaming abstraction**

Why: DStreams are the older Spark Streaming abstraction built around sequences of RDDs. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q3 [Original Practice]

What is the basic purpose of a checkpoint location in a streaming query?

- A) Persist recovery/progress information on durable storage
- B) Increase the number of Java constructors
- C) Convert JSON to XML
- D) Replace every source with a socket

✓ **Answer: A) Persist recovery/progress information on durable storage**

Why: Checkpointing stores streaming metadata/progress and, where needed, state for recovery. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q4 [Original Practice]

A trainer wants a simple live demo where trainees type text into a TCP socket and see Spark process it. Is a socket source reasonable for this purpose?

- A) Yes, for demonstration/testing
- B) No, sockets cannot carry text
- C) Only if the data is first written to JDBC
- D) Only for batch jobs

✓ **Answer: A) Yes, for demonstration/testing**

Why: Socket sources are convenient for demonstrations, but they are not durable production sources. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q5 [Original Practice]

New CSV files are dropped into a watched folder after they are completely written. Which Structured Streaming source can process newly arriving files?

- A) File source with readStream and a defined schema
- B) collect() on a static List
- C) Java FileOutputStream only
- D) LocalDate API

✓ **Answer: A) File source with readStream and a defined schema**

Why: Structured Streaming can monitor directories and incrementally process newly arriving files. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q6 [Original Practice]

A business wants sales totals for each 10-minute interval based on event timestamps. Which concept is appropriate?

- A) Event-time window aggregation
- B) JVM heap only
- C) Method overloading
- D) Object serialization

✓ **Answer: A) Event-time window aggregation**

Why: Windowed aggregation groups events into time ranges such as 10-minute windows. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q7 [Original Practice]

Events may arrive several minutes late, but Spark should eventually stop retaining old event-time state. Which feature addresses accepted lateness and state cleanup?

- A) Watermark
- B) Autoboxing
- C) JDBC commit
- D) RDD parallelize

✓ **Answer: A) Watermark**

Why: Watermarks define a threshold for late event-time data and help bound state for supported stateful operations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q8 [Original Practice]

A streaming query restarts after a driver failure. Which design is important if it should continue using previously recorded progress?

- A) Restart with the same durable checkpoint location when the query is compatible
- B) Use only executor cache
- C) Always create a brand-new checkpoint
- D) Store offsets only in a local mutable variable

✓ **Answer: A) Restart with the same durable checkpoint location when the query is compatible**

Why: A durable checkpoint can preserve progress and state needed for compatible query recovery. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q9 [Original Practice]

An upstream process writes large files slowly. How can it reduce the chance that Spark sees partially written input files in a watched folder?

- A) Write elsewhere and atomically move/rename completed files into the watched location
- B) Let Spark read while files are still being written
- C) Delete files before they finish
- D) Store filenames only in driver memory

✓ **Answer: A) Write elsewhere and atomically move/rename completed files into the watched location**

Why: Publishing only completed files into the watched directory is a common safe pattern for file-stream ingestion. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q10 [Original Practice]

A query uses event-time windows and allows events up to 10 minutes late. Why is a 5-minute processing trigger not a replacement for a 10-minute watermark?

- A) The trigger controls how often processing runs, while the watermark relates to event-time lateness/state
- B) The trigger changes event timestamps
- C) The watermark only controls executor count
- D) They are identical settings

✓ **Answer: A) The trigger controls how often processing runs, while the watermark relates to event-time lateness/state**

Why: Trigger cadence and event-time lateness are different concepts: trigger controls processing frequency; watermark controls late-data/state semantics. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q11 [Direct]

Which Spark streaming API is the modern DataFrame/Dataset-style abstraction?

- A) Structured Streaming
- B) DStreams
- C) FileInputStream
- D) JDBC

✓ **Answer: B) DStreams**

Why: Structured Streaming uses the structured DataFrame/Dataset model. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q12 [Direct]

What are DStreams?

- A) Older RDD-based Spark Streaming abstraction
- B) Parquet partitions
- C) JDBC transactions
- D) Java interfaces

✓ **Answer: B) Parquet partitions**

Why: DStreams represent the older micro-batch/RDD-oriented streaming abstraction. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q13 [Direct]

What is the main purpose of a durable checkpoint location for a streaming query?

- A) Preserve progress/state information for compatible recovery
- B) Increase Java heap automatically
- C) Replace the source
- D) Convert JSON to Parquet

✓ **Answer: B) Increase Java heap automatically**

Why: Checkpoints store metadata/progress and, where applicable, state needed for recovery. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q14 [Direct]

What does a streaming window do?

- A) Groups recent events over a time interval
- B) Compiles Java
- C) Creates a JDBC pool
- D) Removes all late data

✓ **Answer: B) Compiles Java**

Why: Windows define time-based groups for streaming aggregations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q15 [Direct]

What problem do watermarks address?

- A) Event-time lateness and bounded state retention
- B) Java autoboxing
- C) JDBC commits
- D) Class inheritance

✓ **Answer: B) Java autoboxing**

Why: Watermarks define how late event-time data is treated and help bound state for supported operations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q16 [Conceptual]

Why is a socket source generally considered a demo/testing source?

- A) It is convenient but not a durable replayable production source
- B) It cannot transmit strings
- C) It is a database
- D) It always writes checkpoints

✓ **Answer: B) It cannot transmit strings**

Why: Socket input is useful for demonstrations but lacks production durability characteristics. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q17 [Conceptual]

What is the difference between window length and slide interval?

- A) Length is the history included; slide is how often the window advances/updates
- B) They are always identical
- C) Slide controls executor memory only
- D) Length controls Java compilation

✓ **Answer: B) They are always identical**

Why: A 30-minute window with a 5-minute slide includes 30 minutes of history and updates every 5 minutes. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q18 [Conceptual]

Why is a durable checkpoint different from cache?

- A) Checkpoint supports recovery/progress; cache primarily supports reuse/performance
- B) Cache always survives driver failure
- C) Checkpoint only speeds repeated queries
- D) They are identical

✓ **Answer: B) Cache always survives driver failure**

Why: Cache/persist is mainly a reuse optimization; checkpoint is for durable recovery/progress/state semantics. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q19 [Conceptual]

Why should completed files be published atomically into a streaming directory?

- A) To reduce the chance Spark sees partially written files
- B) To increase Java inheritance
- C) To disable watermarks
- D) To make every file a database

✓ **Answer: B) To increase Java inheritance**

Why: Writing elsewhere and atomically moving completed files helps file-streaming ingestion see complete inputs. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q20 [Scenario]

New JSON files are dropped into a watched directory after they are fully written. Which source pattern fits?

- A) Structured Streaming file source with an appropriate schema
- B) Socket source only
- C) RDD collect
- D) JDBC transaction

✓ **Answer: B) Socket source only**

Why: Structured Streaming can incrementally process newly arriving files. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q21 [Scenario]

A streaming query must recover progress after a driver failure. What should be configured durably?

- A) Checkpoint location
- B) Executor cache only
- C) A local mutable variable
- D) Java stack frame

✓ **Answer: B) Executor cache only**

Why: Durable checkpoints preserve progress/state needed for compatible query recovery. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q22 [Scenario]

A business wants rolling sales over 10-minute event-time intervals. Which concept is central?

- A) Event-time window
- B) Autoboxing
- C) HashSet
- D) JDBC rollback

✓ **Answer: B) Autoboxing**

Why: Windowed aggregation groups events by event-time intervals. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q23 [Scenario]

Events may arrive 8 minutes late and state should not be kept forever. Which feature is relevant?

- A) Watermark
- B) map
- C) commit
- D) Pattern

✓ **Answer: B) map**

Why: Watermarks express event-time lateness tolerance and help bound state. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q24 [Scenario]

A trainer wants trainees to type messages into a TCP connection for a simple Spark streaming demo. Which source can be reasonable?

- A) Socket source
- B) Durable checkpoint as a source
- C) JDBC rollback
- D) Parquet pruning

✓ **Answer: B) Durable checkpoint as a source**

Why: Socket sources are convenient for simple demonstrations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q25 [Scenario]

A trigger runs every 5 minutes, while the requirement allows events 10 minutes late. Which statement is correct?

- A) Trigger controls processing cadence; watermark controls event-time lateness semantics
- B) Trigger and watermark are the same
- C) Trigger changes event timestamps
- D) Watermark controls executor count

✓ **Answer: B) Trigger and watermark are the same**

Why: These settings solve different problems: scheduling frequency versus late-event/state handling. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q26 [Conceptual]

Which architecture is generally preferred for a new DataFrame-style streaming application?

- A) Structured Streaming
- B) DStreams unless forced
- C) Socket API only
- D) Java serialization

✓ **Answer: B) DStreams unless forced**

Why: The preparation notes position Structured Streaming as the modern structured API. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q27 [Conceptual]

What should you associate with a longer watermark tolerance?

- A) More late data may be accepted, but state may be retained longer
- B) Less state always
- C) No event-time semantics
- D) Automatic JDBC pooling

✓ **Answer: B) Less state always**

Why: Greater lateness tolerance can increase state retention for stateful operations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q28 [Direct]

A new DataFrame-oriented Spark streaming application is being designed. Which API is the expected choice?

- A) DStreams only
- B) Structured Streaming
- C) java.util.regex
- D) Iterator

✓ **Answer: B) Structured Streaming**

Why: Structured Streaming is the modern DataFrame-oriented streaming API. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q29 [Scenario]

A streaming query restarts after driver failure. Which artifact is intended to preserve recoverable progress/state information?

- A) Durable checkpoint
- B) collect()
- C) LocalDate
- D) List

✓ **Answer: A) Durable checkpoint**

Why: Checkpoint data stores progress/state needed for recovery and must be durable. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q30 [Window reasoning]

A 10-minute window slides every 2 minutes. Which statement is correct?

- A) The window contains 2 minutes of history
- B) The window spans 10 minutes and advances every 2 minutes
- C) Watermark equals 2 minutes automatically
- D) Trigger must be 10 minutes

✓ **Answer: B) The window spans 10 minutes and advances every 2 minutes**

Why: Window length describes the amount of event-time history; slide interval describes how often the window advances. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property before selecting the option.

Q31 [Direct]

Which mechanism primarily addresses late event-time data and state cleanup?

- A) Watermark
- B) JDK
- C) unionByName
- D) Connection pool

✓ **Answer: A) Watermark**

Why: Watermarks establish a lateness threshold and help bound state for event-time operations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q32 [Scenario]

Why is a socket source commonly described as a demo/testing source?

- A) It is always durable and replayable
- B) It is ephemeral and not designed as a durable production source
- C) It automatically checkpoints data
- D) It is a JDBC database

✓ **Answer: B) It is ephemeral and not designed as a durable production source**

Why: Socket input is convenient for demonstrations but lacks the durable replay characteristics expected of production ingestion. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

13. Java OOPs / APIs / Collections

Difficult difficulty • 2 official MCQs • 4 official marks • Expanded practice below

What this section is really testing

The final difficult Java MCQ area in the supplied bifurcation is best treated as an integrated trap section. The handbook's stretch material emphasizes race conditions, deadlocks, concurrent collections, `CompletableFuture`, JDBC transactions and connection pooling, while also revisiting Java API/collection distinctions.

Deep Concept Notes

Race conditions

A race condition occurs when multiple threads access shared mutable state and the result depends on timing/interleaving. A classic lost-update scenario occurs when two threads read the same value, both compute an update and then overwrite each other.

Deadlock

Deadlock occurs when threads wait indefinitely for locks held by one another, commonly through circular wait. A simple prevention strategy is consistent global lock ordering: acquire multiple locks in the same order everywhere.

ConcurrentHashMap

`ConcurrentHashMap` is designed for concurrent map access and updates. At this assessment depth, choose it over an ordinary `HashMap` when multiple threads need concurrent access. Compound updates should use appropriate atomic map methods rather than assuming every multi-step sequence is automatically atomic.

CopyOnWriteArrayList

`CopyOnWriteArrayList` creates a fresh internal copy on modification. This makes iteration safe and read-friendly but makes frequent writes expensive. The exam trigger is "many reads, very few writes," such as listener collections.

CompletableFuture

`CompletableFuture` represents an asynchronous computation that can be chained or combined. It is used when work can proceed asynchronously or when dependent asynchronous results must be composed.

JDBC transactions

A transaction groups related database changes so they succeed or fail as a unit. With auto-commit disabled, successful work is committed; failure can trigger rollback. Transfer/order scenarios with all-or-nothing requirements are classic transaction questions.

Connection pooling

A connection pool reuses physical database connections instead of repeatedly creating new ones. Borrow → use → close/return is the normal lifecycle. Closing a pooled connection typically returns it to the pool rather than destroying the pool itself.

Exam Trigger Map

- Race = timing-dependent shared-state bug.
- Deadlock = circular waiting for locks.
- Consistent lock order reduces circular-wait risk.
- `CopyOnWriteArrayList` = read-heavy, write-rare.

- commit makes transaction changes permanent; rollback undoes uncommitted work.
- Connection pool reduces repeated physical connection creation.

FAST SIGNAL	THINK OF
Trap	Race = timing-dependent shared-state bug.
Trap	Deadlock = circular waiting for locks.
Trap	Consistent lock order reduces circular-wait risk.
Trap	CopyOnWriteArrayList = read-heavy, write-rare.
Trap	commit makes transaction changes permanent; rollback undoes uncommitted work.
Trap	Connection pool reduces repeated physical connection creation.

MCQ Training Bank — 31 Questions

Question styles are intentionally mixed: direct recall, scenario-based, conceptual reasoning, comparison, trap questions, classification, cardinality/output reasoning and performance/architecture reasoning where appropriate.

Q1 [Original Practice]

Two threads update the same shared counter and some increments are lost. Which concurrency problem is this?

- A) Race condition
- B) Method overloading
- C) Schema inference
- D) RDD lineage

✓ **Answer: A) Race condition**

Why: A race condition occurs when concurrent operations on shared state produce results that depend on timing/interleaving. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q2 [Original Practice]

Thread A holds Lock1 and waits for Lock2 while Thread B holds Lock2 and waits for Lock1 indefinitely. What problem is illustrated?

- A) Deadlock
- B) Autoboxing
- C) Serialization
- D) Column pruning

✓ **Answer: A) Deadlock**

Why: The threads are in a circular wait, which is the classic deadlock pattern. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q3 [Original Practice]

Creating an order and reducing inventory must either both succeed or both fail. Which JDBC transaction approach is appropriate?

- A) Disable auto-commit, commit after both succeed, rollback on failure
- B) Commit after the first statement regardless of later errors
- C) Use a different connection for each statement with no coordination
- D) Call collect() after each SQL statement

✓ **Answer: A) Disable auto-commit, commit after both succeed, rollback on failure**

Why: A single transaction with commit/rollback keeps related database changes atomic. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q4 [Original Practice]

Two fields must be updated together so readers never observe a half-updated state. What concept can protect the compound operation?

- A) Use the same shared lock/synchronization around related updates and reads
- B) Use two unrelated local locks created per call
- C) Mark only one field static
- D) Use HashSet instead of synchronization

✓ **Answer: A) Use the same shared lock/synchronization around related updates and reads**

Why: Compound invariants require coordinated synchronization, not merely separate operations on each field. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q5 [Original Practice]

Many threads need to read and update a shared map. Which Java collection is designed for concurrent access better than a plain HashMap without external synchronization?

- A) ConcurrentHashMap
- B) ArrayList
- C) TreeSet only
- D) StringBuilder

✓ **Answer: A) ConcurrentHashMap**

Why: ConcurrentHashMap is a concurrent map implementation designed for multi-threaded access. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q6 [Original Practice]

A service wants to represent a result that will be completed asynchronously and then apply another non-blocking step. Which Java API is designed for this style?

- A) CompletableFuture
- B) FileInputStream
- C) LocalDate
- D) HashSet

✓ **Answer: A) CompletableFuture**

Why: CompletableFuture models asynchronous computations and supports chaining stages. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q7 [Original Practice]

A listener list is read very frequently and modified only occasionally. Which concurrent list is known for snapshot-style iteration with relatively expensive writes?

- A) CopyOnWriteArrayList
- B) LinkedHashMap
- C) ArrayDeque
- D) String[]

✓ **Answer: A) CopyOnWriteArrayList**

Why: CopyOnWriteArrayList is suitable for read-heavy scenarios where writes are rare and snapshot iteration is useful. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q8 [Original Practice]

Why do server applications commonly use a JDBC connection pool?

- A) To reuse a limited set of database connections instead of opening a new physical connection for every request
- B) To convert SQL into Scala
- C) To guarantee every query is fast
- D) To replace database transactions

✓ **Answer: A) To reuse a limited set of database connections instead of opening a new physical connection for every request**

Why: Connection pooling reduces connection-creation overhead and manages a reusable pool of connections. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q9 [Original Practice]

Transfers can lock two Account objects. One thread locks account 10 then 20, while another may lock 20 then 10. Which simple design reduces deadlock risk while keeping per-account locks?

- A) Acquire required account locks in a consistent deterministic order such as ascending account ID
- B) Always lock the source first regardless of ID
- C) Create a brand-new lock object for each transfer
- D) Hold the first lock while repeatedly sleeping forever

✓ **Answer: A) Acquire required account locks in a consistent deterministic order such as ascending account ID**

Why: A consistent lock ordering removes the circular-wait pattern that can cause deadlock. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: answer from the concept, then verify against the explanation.

Q10 [Direct]

What is a race condition?

- A) A timing-dependent result caused by concurrent access to shared mutable state
- B) A compile-time overload
- C) A file-format mismatch
- D) A database schema

✓ **Answer: B) A compile-time overload**

Why: Race conditions arise when interleavings of concurrent operations affect the result. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q11 [Direct]

What is deadlock?

- A) Threads wait indefinitely in a circular dependency for resources
- B) A fast asynchronous computation
- C) A successful transaction
- D) A hash collision

✓ **Answer: B) A fast asynchronous computation**

Why: Deadlock is characterized by circular waiting and no progress. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q12 [Direct]

Which collection is designed for concurrent Map access?

- A) ConcurrentHashMap
- B) ArrayList
- C) HashSet only
- D) StringBuilder

✓ **Answer: B) ArrayList**

Why: ConcurrentHashMap supports concurrent map operations without requiring external synchronization for every simple access. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q13 [Direct]

When is CopyOnWriteArrayList especially suitable?

- A) Many reads and infrequent writes
- B) Constant heavy writes
- C) Database transactions only
- D) Sorting primitive arrays

✓ **Answer: B) Constant heavy writes**

Why: Writes copy the underlying array, so it is most suitable when reads dominate. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q14 [Direct]

Which API models an asynchronous computation that can be chained?

- A) CompletableFuture
- B) FileReader
- C) LocalDate
- D) HashMap

✓ **Answer: B) FileReader**

Why: CompletableFuture represents asynchronous results and supports dependent stages. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q15 [Conceptual]

What is the simplest deadlock-prevention rule for code that must acquire multiple locks?

- A) Always acquire them in a consistent global order
- B) Acquire them randomly
- C) Sleep after every lock
- D) Use a different order in each thread

✓ **Answer: B) Acquire them randomly**

Why: A consistent order removes the circular-wait pattern caused by inconsistent acquisition. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q16 [Conceptual]

Why is a plain HashMap risky for unsynchronized concurrent updates?

- A) It is not designed as a general concurrent map implementation
- B) It is immutable
- C) It cannot store Strings
- D) It always sorts keys

✓ **Answer: B) It is immutable**

Why: ConcurrentHashMap exists for concurrent map access; a plain HashMap does not provide that guarantee. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q17 [Conceptual]

What does commit() do in a JDBC transaction?

- A) Makes the transaction changes permanent
- B) Undoes uncommitted changes
- C) Closes every database
- D) Starts a new thread

✓ **Answer: B) Undoes uncommitted changes**

Why: commit finalizes the current transaction changes. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q18 [Conceptual]

What does rollback() do?

- A) Undoes uncommitted transaction changes
- B) Makes all changes permanent
- C) Creates a connection pool
- D) Compiles SQL

✓ **Answer: B) Makes all changes permanent**

Why: rollback returns the transaction to its pre-transaction state for uncommitted work. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q19 [Conceptual]

Why is connection pooling useful?

- A) It reuses database connections and reduces connection creation overhead
- B) It replaces SQL
- C) It prevents all deadlocks
- D) It converts JDBC to RDD

✓ **Answer: B) It replaces SQL**

Why: A pool keeps reusable connections so each request need not create a new physical connection. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q20 [Scenario]

Two threads update the same counter and sometimes one increment disappears. What should you investigate first?

- A) Race condition in a shared read-modify-write operation
- B) Parquet pruning
- C) Regex matching
- D) Method overloading

✓ **Answer: B) Parquet pruning**

Why: Lost updates are a classic sign of an unsynchronized race on shared mutable state. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q21 [Scenario]

Thread A holds Lock X and requests Lock Y while Thread B holds Y and requests X. What is the problem?

- A) Deadlock
- B) Autoboxing
- C) Type inference
- D) Column pruning

✓ **Answer: B) Autoboxing**

Why: Each thread waits for a resource held by the other, creating circular wait. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q22 [Scenario]

A server processes thousands of requests and each request needs a JDBC connection. Which design reduces repeated physical connection creation?

- A) Connection pool
- B) collect()
- C) CopyOnWriteArrayList only
- D) LocalDate

✓ **Answer: B) collect()**

Why: Pooling reuses a managed set of database connections. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q23 [Scenario]

An order creation updates inventory and order status; if either fails, neither change should remain. What is the correct transaction strategy?

- A) Disable auto-commit, commit after both succeed, rollback on failure
- B) Commit after the first update
- C) Use unrelated connections with no transaction
- D) Ignore exceptions

✓ **Answer: B) Commit after the first update**

Why: A transaction provides all-or-nothing semantics when commit/rollback are used correctly. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q24 [Scenario]

A listener registry is read by many threads thousands of times per second but changes only once per hour. Which collection is a strong conceptual fit?

- A) CopyOnWriteArrayList
- B) ArrayList with unsynchronized writes
- C) HashMap
- D) String

✓ **Answer: B) ArrayList with unsynchronized writes**

Why: Copy-on-write favors read-heavy, write-rarely workloads. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q25 [Scenario]

A method starts a slow remote call and wants to attach a transformation that runs after the result is available. Which API fits?

- A) CompletableFuture
- B) PrintWriter
- C) Iterator
- D) LocalDate

✓ **Answer: B) PrintWriter**

Why: CompletableFuture supports asynchronous completion and chained stages. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q26 [Scenario]

A transfer locks two Account objects. Which strategy reduces deadlock risk?

- A) Lock accounts in deterministic ID order
- B) Lock source first in one thread and destination first in another
- C) Create random lock order
- D) Hold one lock forever

✓ **Answer: B) Lock source first in one thread and destination first in another**

Why: Consistent lock ordering prevents the inconsistent circular acquisition pattern. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q27 [Concurrency trap]

Thread A locks X then Y; Thread B locks Y then X. What failure mode is this pattern vulnerable to?

- A) Autoboxing
- B) Deadlock
- C) Regex mismatch
- D) Serialization only

✓ **Answer: B) Deadlock**

Why: Each thread can hold one lock while waiting for the other, creating circular wait and potential deadlock. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property before selecting the option.

Q28 [Concurrency scenario]

A counter is incremented by many threads without synchronization, and results vary between runs. What is the strongest diagnosis?

- A) Race condition
- B) Composition
- C) Type inference
- D) Column pruning

✓ **Answer: A) Race condition**

Why: Timing-dependent access to shared mutable state can cause lost updates and nondeterministic results. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify shared state, locking and timing behavior.

Q29 [Scenario]

A listener list is read thousands of times but modified only a few times. Which collection is conceptually suitable?

- A) CopyOnWriteArrayList
- B) ArrayList with uncontrolled concurrent writes
- C) HashMap only
- D) Stack frame

✓ **Answer: A) CopyOnWriteArrayList**

Why: CopyOnWriteArrayList favors read-heavy, write-rare workloads because writes copy the underlying array. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q30 [Scenario]

A money transfer updates account A and account B. If the second update fails, both changes should be undone. Which concept is central?

- A) JDBC transaction with rollback
- B) regex find()
- C) List iteration
- D) HTTP GET

✓ **Answer: A) JDBC transaction with rollback**

Why: Transactions provide all-or-nothing behavior: commit on success and rollback uncommitted work on failure. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q31 [Conceptual]

Why does a connection pool improve database application efficiency?

- A) It avoids repeated creation of physical connections by reusing them
- B) It converts SQL into Java bytecode
- C) It eliminates transactions
- D) It makes every query local

✓ **Answer: A) It avoids repeated creation of physical connections by reusing them**

Why: Opening physical database connections is expensive. A pool reuses connections across requests. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

14. Integrated Mastery — Mixed Concept & Trap Drills

No new syllabus topic; this section deliberately mixes the official areas.

MCQ Training Bank — 39 Questions

Question styles are intentionally mixed: direct recall, scenario-based, conceptual reasoning, comparison, trap questions, classification, cardinality/output reasoning and performance/architecture reasoning where appropriate.

Q1 [Direct]

A question mentions "same method name, different parameter types." Which concept should you identify first?

- A) Overloading
- B) Overriding
- C) Deadlock
- D) Watermark

✓ **Answer: B) Overriding**

Why: Different parameter lists indicate overloading. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q2 [Direct]

A question says "child implementation chosen for a parent reference holding a child object." What is the key concept?

- A) Runtime polymorphism
- B) Autoboxing
- C) Schema inference
- D) Partition pruning

✓ **Answer: B) Autoboxing**

Why: Overridden instance methods are selected using runtime dispatch. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q3 [Direct]

A question says "one input produces multiple outputs and they are flattened." Which answer should come to mind?

- A) flatMap
- B) map
- C) filter
- D) count

✓ **Answer: B) map**

Why: flatMap combines one-to-many mapping with flattening. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q4 [Direct]

A question says "all records are transferred to the driver." Which Spark method is the likely culprit?

- A) collect
- B) filter
- C) map
- D) reduceByKey

✓ **Answer: B) filter**

Why: collect materializes the full result at the driver. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q5 [Direct]

A question says "same key, aggregate values." Which operation is most likely intended for a pair RDD?

- A) reduceByKey
- B) collect
- C) first
- D) cache

✓ **Answer: B) collect**

Why: reduceByKey performs per-key aggregation in a distributed way. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q6 [Conceptual]

A question says "need reuse of an expensive RDD across multiple actions." Which concept is the likely answer?

- A) cache/persist
- B) checkpoint only
- C) collect
- D) watermark

✓ **Answer: B) checkpoint only**

Why: Persistence is a reuse/performance optimization. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q7 [Conceptual]

A question says "restart a streaming query and preserve previous progress." Which concept fits?

- A) Durable checkpoint
- B) cache only
- C) collect
- D) map

✓ **Answer: B) cache only**

Why: Streaming checkpoints preserve compatible progress/state for recovery. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q8 [Conceptual]

A question says "only required columns should be read from Parquet." Which optimization?

- A) Column pruning
- B) Partition pruning
- C) Lock ordering
- D) Type inference

✓ **Answer: B) Partition pruning**

Why: Column pruning reduces unnecessary column I/O. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q9 [Conceptual]

A question says "filter on event_date partition and avoid unrelated folders." Which optimization?

- A) Partition pruning
- B) Column pruning
- C) Autoboxing
- D) Overloading

✓ **Answer: B) Column pruning**

Why: Partition pruning skips irrelevant partitions. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q10 [Conceptual]

A question says "shared mutable counter sometimes loses updates." What should you identify?

- A) Race condition
- B) Deadlock
- C) Overloading
- D) Checkpoint

✓ **Answer: B) Deadlock**

Why: Lost updates caused by timing/interleaving indicate a race condition. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q11 [Scenario]

A batch pipeline reads structured Parquet, filters rows, aggregates by region, and writes results. Which abstraction is the natural default?

- A) DataFrame
- B) Local Array only
- C) Socket source
- D) Thread

✓ **Answer: B) Local Array only**

Why: Structured analytics with named columns and relational operations is a DataFrame use case. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q12 [Scenario]

A database read is too slow and has a suitable numeric partition key. Which family of settings should you investigate?

- A) JDBC partitionColumn/bounds/numPartitions
- B) Java regex Pattern
- C) Scala guard
- D) CopyOnWriteArrayList

✓ **Answer: B) Java regex Pattern**

Why: Partitioned JDBC reads can parallelize source access when a suitable key and bounds exist. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q13 [Scenario]

A file-streaming job occasionally reads a file while it is still being written. Which upstream change is useful?

- A) Publish completed files atomically into the watched directory
- B) Call collect()
- C) Increase Java heap only
- D) Replace DataFrame with HashMap

✓ **Answer: B) Call collect()**

Why: Atomic publication reduces partial-file visibility. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q14 [Scenario]

A validation function needs to return either a detailed error or a valid object. Which Scala type fits?

- A) Either[Error, Value]
- B) Option only
- C) Tuple only
- D) Array only

✓ **Answer: B) Option only**

Why: Either can carry explicit error information or a successful value. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q15 [Scenario]

A service needs an asynchronous result and a dependent transformation without manually managing a thread. Which API is relevant?

- A) CompletableFuture
- B) LocalDate
- C) PrintWriter
- D) HashSet

✓ **Answer: B) LocalDate**

Why: CompletableFuture models asynchronous completion and chained stages. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q16 [Scenario]

A transfer operation may lock two accounts in different orders in different threads. Which prevention rule is most useful?

- A) Acquire locks in a consistent deterministic order
- B) Randomize lock order
- C) Sleep after first lock
- D) Create a fresh lock for each call

✓ **Answer: B) Randomize lock order**

Why: Consistent ordering reduces circular wait and therefore deadlock risk. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q17 [Scenario]

A report selects five columns from a 200-column Parquet dataset and filters a partition column. Which pair of optimizations should you recognize?

- A) Column pruning and partition pruning
- B) Autoboxing and overloading
- C) Rollback and commit
- D) Race and deadlock

✓ **Answer: B) Autoboxing and overloading**

Why: The projection can enable column pruning and the partition filter can enable partition pruning. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q18 [Direct]

Which pair is matched correctly?

- A) matches=whole input, find=subsequence
- B) matches=subsequence, find=whole input
- C) both always identical
- D) both are file APIs

✓ **Answer: B) matches=subsequence, find=whole input**

Why: matches validates the whole input; find searches within it. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q19 [Direct]

Which pair is matched correctly?

- A) cache=reuse/performance, checkpoint=recovery/progress
- B) cache=database commit, checkpoint=regex
- C) cache=watermark, checkpoint=overload
- D) both are identical

✓ **Answer: B) cache=database commit, checkpoint=regex**

Why: Persistence is for reuse; checkpointing supports recovery/progress/state. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q20 [Direct]

Which pair is matched correctly?

- A) race=timing-dependent wrong result, deadlock=circular waiting
- B) race=circular waiting, deadlock=wrong result
- C) both mean database rollback
- D) both mean shuffle

✓ **Answer: B) race=circular waiting, deadlock=wrong result**

Why: Race and deadlock are distinct concurrency failure modes. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q21 [Direct]

Which pair is matched correctly?

- A) map=transform, filter=select, flatMap=expand+flatten
- B) map=select, filter=expand, flatMap=compile
- C) all three are actions
- D) all three are JDBC operations

✓ **Answer: B) map=select, filter=expand, flatMap=compile**

Why: These are the core functional transformation meanings. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q22 [Direct]

Which pair is matched correctly?

- A) List=ordered duplicates, Set=unique, Map=key/value
- B) List=key/value, Set=ordered duplicates, Map=unique only
- C) all are identical
- D) Set always indexed

✓ **Answer: B) List=key/value, Set=ordered duplicates, Map=unique only**

Why: The three collection abstractions serve different data-modeling needs. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: identify the defining property being tested.

Q23 [Conceptual]

Which comparison is correct?

- A) overload=compile-time parameter selection; override=runtime child behavior
- B) overload=runtime child behavior; override=parameter selection
- C) both require different return types only
- D) neither uses parameters

✓ **Answer: B) overload=runtime child behavior; override=parameter selection**

Why: Overloading is distinguished by parameters; overriding replaces inherited behavior. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q24 [Conceptual]

Which comparison is correct?

- A) explicit schema gives control; inferSchema asks Spark to infer types
- B) explicit schema means no types; inferSchema means manual types
- C) both guarantee clean data
- D) inferSchema disables parsing

✓ **Answer: B) explicit schema means no types; inferSchema means manual types**

Why: Explicit schema controls expected structure, while inferSchema derives likely types from data. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q25 [Conceptual]

Which comparison is correct?

- A) column pruning reduces columns read; partition pruning reduces partitions scanned
- B) both delete physical data
- C) column pruning is JDBC locking
- D) partition pruning is regex

✓ **Answer: B) both delete physical data**

Why: They reduce different dimensions of unnecessary I/O. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q26 [Conceptual]

Which comparison is correct?

- A) window groups event-time history; watermark handles lateness/state bounds
- B) window handles JDBC commits; watermark handles inheritance
- C) both mean trigger interval
- D) watermark creates a List

✓ **Answer: B) window handles JDBC commits; watermark handles inheritance**

Why: Windowing and watermarking solve different streaming problems. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q27 [Conceptual]

Which comparison is correct?

- A) commit finalizes transaction; rollback undoes uncommitted work
- B) commit undoes; rollback finalizes
- C) both close JDBC pools
- D) both create threads

✓ **Answer: B) commit undoes; rollback finalizes**

Why: Commit and rollback are opposite transaction-finalization paths. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q28 [Scenario]

A question says "new files arrive continuously, process them incrementally, use DataFrame-style operations." Which answer is strongest?

- A) Structured Streaming file source
- B) DStream socket only
- C) Java byte stream
- D) HashMap

✓ **Answer: B) DStream socket only**

Why: The clues point to Structured Streaming with a file source. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q29 [Scenario]

A question says "the same expensive RDD is used in three actions and recomputation is wasting time." What should you choose?

- A) Persist/cache
- B) Checkpoint only because it is faster by definition
- C) collect
- D) watermark

✓ **Answer: B) Checkpoint only because it is faster by definition**

Why: Persisting reused computation is the standard performance-oriented response. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q30 [Scenario]

A question says "a query contains an Exchange after aggregation." What should you infer?

- A) Data redistribution/shuffle is involved
- B) The query is doing regex
- C) A Java constructor ran
- D) The database committed

✓ **Answer: B) The query is doing regex**

Why: Exchange commonly indicates redistribution needed by operations such as aggregations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q31 [Scenario]

A question says "only 10% of rows need an expensive custom UDF." What ordering is generally sensible?

- A) Filter the 90% out first when logically valid
- B) UDF all rows first
- C) collect all rows
- D) disable partitioning

✓ **Answer: B) UDF all rows first**

Why: Filtering before expensive custom work reduces the number of UDF evaluations. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q32 [Scenario]

A question says "multiple threads frequently read a listener list; writes are rare." Which collection is a strong fit?

- A) CopyOnWriteArrayList
- B) HashMap
- C) ArrayDeque
- D) StringBuilder

✓ **Answer: B) HashMap**

Why: Copy-on-write collections are designed for read-heavy, write-rarely scenarios. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q33 [Scenario]

A question says "two related database updates must both succeed or both fail." Which concept is central?

- A) JDBC transaction with commit/rollback
- B) Regex matching
- C) RDD lineage
- D) Scala tuple

✓ **Answer: B) Regex matching**

Why: A transaction provides atomic all-or-nothing semantics when used with commit/rollback. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q34 [Conceptual]

A question says "a parent reference points to a child and the child method runs." Which two ideas are linked?

- A) Overriding and runtime polymorphism
- B) Overloading and autoboxing
- C) Serialization and pruning
- D) Watermark and window

✓ **Answer: B) Overloading and autoboxing**

Why: The child method is selected through runtime dispatch because the method is overridden. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: focus on the mechanism's purpose, not a memorized example.

Q35 [Comparison]

Which pairing is correctly matched?

- A) matches()—search any substring; find()—whole input
- B) map—one output per input; flatMap—zero/many outputs plus flattening
- C) Set—key/value pairs; Map—unique values only
- D) watermark—processing schedule; trigger—late data threshold

✓ **Answer: B) map—one output per input; flatMap—zero/many outputs plus flattening**

Why: map is one-to-one by cardinality; flatMap can expand/flatten. The other pairings reverse important distinctions. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: eliminate options that swap the two concepts.

Q36 [Comparison]

Which pairing is correctly matched?

- A) cache—durable streaming recovery; checkpoint—temporary reuse
- B) partition pruning—skip unused partitions; column pruning—skip unused columns
- C) List—unique elements; Set—duplicates
- D) overloading—runtime dispatch; overriding—compile-time parameter selection

✓ **Answer: B) partition pruning—skip unused partitions; column pruning—skip unused columns**

Why: Partition pruning reduces partitions scanned; column pruning reduces columns read. The other choices reverse concepts. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: eliminate options that swap the two concepts.

Q37 [Scenario]

Which scenario most strongly signals a wide/shuffle-related Spark operation?

- A) Applying map to each element
- B) Filtering each record locally
- C) Grouping/aggregating by key across partitions
- D) Reading a local integer variable

✓ **Answer: C) Grouping/aggregating by key across partitions**

Why: Per-key grouping/aggregation can require records with the same key to meet, causing redistribution/shuffle. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: translate the scenario into a requirement before looking at the options.

Q38 [Meta-reasoning]

Which statement is the best overall exam strategy for an unfamiliar scenario?

- A) Pick the longest option
- B) Identify the requirement/trigger word first, then map it to the mechanism
- C) Always choose UDF
- D) Always choose collect()

✓ **Answer: B) Identify the requirement/trigger word first, then map it to the mechanism**

Why: The handbook repeatedly emphasizes conceptual recognition: requirement → mechanism. Trigger words make unfamiliar wording easier to decode. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: use the requirement/trigger word instead of the option's wording length.

Q39 [Trap]

Which statement is correct?

- A) Explicit schema guarantees valid source data
- B) A watermark determines how often a streaming query runs
- C) A final method cannot be overridden
- D) collect() is safe for arbitrarily large RDDs

✓ **Answer: C) A final method cannot be overridden**

Why: A final method cannot be overridden. The other statements confuse schema with validation, watermark with trigger, and collect with driver memory. The key is to match the requirement in the question to the mechanism's defining purpose.

Exam clue: compare the tempting alternatives against the exact definition.

Final Rapid-Revision: High-Confusion Pairs

PAIR	ONE-LINE DISTINCTION
JDK vs JVM	JDK develops/compiles; JVM executes bytecode.
List vs Set vs Map	List → ordered/duplicates; Set → unique elements; Map → key/value association.
Stack vs Heap	Stack frames hold local variables/references; objects are generally allocated on heap.
Overloading vs Overriding	Overloading → different parameter list/compile-time selection; overriding → child replacement/runtime dispatch.
Inheritance vs Composition	is-a → inheritance; has-a → composition.
matches vs find	matches → whole input; find → matching subsequence.
LocalDate vs LocalDateTime vs ZonedDateTime	Date only; date+time; date+time+zone.
val vs var	val cannot be rebound; var can be reassigned.
List vs Array in Scala	List immutable; Array indexed and mutable.
map vs filter vs flatMap	map transforms one-to-one; filter keeps/discards; flatMap expands and flattens.
Transformation vs Action	Transformation is lazy; action triggers execution.
Lineage vs Checkpoint	Lineage enables recomputation; checkpoint materializes recovery state.
cache/persist vs checkpoint	Persistence is for reuse/performance; checkpoint is for durable recovery/state.
collect vs distributed processing	collect moves everything to driver; distributed operations keep work distributed.
DataFrame vs RDD	DataFrame has named columns/schema and optimizer visibility; RDD is lower-level object/record abstraction.
Explicit schema vs inferSchema	Explicit schema is predictable; inferSchema guesses from input.
unionByName vs positional union	unionByName aligns columns by name.
Column pruning vs partition pruning	Column pruning reads fewer columns; partition pruning scans fewer partitions.
Built-in vs UDF	Prefer built-in when equivalent because Spark can understand/optimize it.
Exchange vs local processing	Exchange signals redistribution/shuffle.
Window vs slide	Window length = history included; slide = how often the window advances.

PAIR	ONE-LINE DISTINCTION
Watermark vs trigger	Watermark manages lateness/state; trigger controls processing timing.
Race vs deadlock	Race = timing-dependent shared-state result; deadlock = circular waiting.
CopyOnWriteArrayList	Many reads + rare writes.
Commit vs rollback	Commit makes transaction changes permanent; rollback undoes uncommitted work.
Option vs Try vs Either	Option = absence; Try = exception success/failure; Either = success plus explicit alternate/error information.

Final MCQ Readiness Checklist

- Can I explain the difference between JDK, JRE and JVM without looking?
- Can I choose List, Set or Map from a business requirement?
- Can I distinguish overloading, overriding, inheritance and composition from scenarios?
- Can I choose byte streams vs character streams and explain try-with-resources?
- Can I distinguish matches() from find() and LocalDate/LocalDateTime/ZonedDateTime?
- Can I predict val/var and immutable List behavior in short Scala snippets?
- Can I classify map/filter/flatMap and explain their output cardinality?
- Can I classify RDD transformations vs actions and explain lazy evaluation?
- Can I explain collect(), lineage, reduceByKey and persistence without memorizing a sentence?
- Can I distinguish Option, Try and Either from scenarios?
- Can I explain DataFrame schema, unionByName and allowMissingColumns?
- Can I recognize Exchange/shuffle and explain why built-ins are usually preferred over UDFs?
- Can I distinguish JDBC read partitioning, column pruning and partition pruning?
- Can I distinguish Structured Streaming, checkpoint, watermark, window and trigger?
- Can I diagnose race vs deadlock and choose transaction/pooling/concurrent-collection concepts?
- Can I answer the same concept when the wording is indirect or the options are deliberately similar?

Practice bank size: 384 MCQs. The bank combines the earlier 100-question set, the expanded question bank, and additional scenario/trap/comparison questions. No separate coding-question section is included.

Best final method: learn each topic → attempt without looking → mark uncertain questions → review the explanation → redo only the wrong/uncertain questions → finish with the mixed mastery section.